



北京理工大学珠海学院

BEIJING INSTITUTE OF TECHNOLOGY, ZHUHAI

基于机器学习的信用卡欺诈检测研究

学 院： 数理与土木工程学院

课 程： 大数据与金融

班 级： 数据科学与大数据技术 2 班

姓 名： 肖宇涵 学 号： 211205101786

中国·珠海

2024 年 6 月 3 日

基于机器学习的信用卡欺诈检测研究

摘 要

信用卡欺诈已成为金融机构日益严重的威胁。尽管现有的信用卡欺诈检测算法在防范欺诈方面取得了一定的进展，但它们仍面临许多问题，如高误报率和漏报率、新型欺诈技术的出现以及数据高度不平衡等问题。

为解决这一问题，本研究通过系统化方法提升信用卡欺诈检测的准确性。首先，采用数据标准化技术，如 Z-score 和 Min-Max 标准化，以消除特征量纲差异。其次，利用 SMOTE 和 ADASYN 进行数据上采样，解决数据不平衡问题。然后，评估孤立森林和局部异常因子两种异常检测方法。最终，选择并优化逻辑回归、随机森林和 XGBoost 三种分类模型，通过超参数调优显著提升模型性能。实验结果表明，XGBoost 模型在 AUPRC、准确率、召回率和 F1 分数方面表现出色，尤其在处理不平衡数据集时效果显著。本研究还通过特征重要性分析和学习曲线验证了模型的有效性和稳定性，为信用卡欺诈检测提供了可靠的技术支持。

关键词：信用卡欺诈 异常检测 AUPRC XGBoost

目 录

1	引言	1
1.1	问题背景	1
1.2	问题描述	1
1.3	研究目的	1
2	文献综述	2
2.1	统计方法和传统技术	2
2.2	机器学习方法	2
2.3	深度学习方法	2
2.4	综合策略	2
2.5	创新方法	2
3	算法介绍	3
3.1	数据标准化	3
3.1.1	为什么需要进行数据标准化?	3
3.1.2	采用什么方式进行数据标准化?	3
3.1.3	数据标准化对模型效果 (AUPRC) 的影响	3
3.2	数据上采样	3
3.2.1	为什么需要进行上采样?	3
3.2.2	上采样的详细步骤	4
3.2.3	上采样对模型效果 (AUPRC) 的影响	4
3.3	异常检测	4
3.3.1	为什么需要进行异常检测?	4
3.3.2	异常检测的方法	4
3.3.3	孤立森林算法原理	5
3.3.4	局部异常因子算法原理	5
3.3.5	异常检测对模型效果 (AUPRC) 的影响	6
3.4	模型构建	7
3.4.1	模型选择	7
3.5	逻辑回归	7
3.5.1	原理	7
3.6	随机森林 (Random Forest)	9
3.6.1	原理	9
3.6.2	构建过程	9

3.7	XGBoost	10
3.7.1	原理	10
3.7.2	超参数选择及其意义	11
3.7.3	模型效果 (AUPRC) 的影响	12
4	数据处理	12
4.1	基本介绍	12
4.2	探索性数据分析	12
4.2.1	交易时间分布	13
4.2.2	交易金额分布	13
4.2.3	查看数据标签分布	14
4.3	数据标准化	14
4.3.1	Z-score 标准化	14
4.3.2	min-max 标准化	15
4.3.3	数据标准化对模型效果 (AUPRC) 的影响	16
4.3.4	逻辑回归模型	17
4.3.5	随机森林模型	18
4.3.6	结论	18
4.4	数据上采样	18
4.4.1	实验结果	19
4.4.2	逻辑回归模型	19
4.4.3	随机森林模型	19
4.4.4	结果分析	20
4.4.5	结论	20
5	异常检测模型构建与分析	20
5.1	异常检测	20
5.1.1	异常检测对 AUPRC 的影响	20
5.1.2	实验结果	21
5.1.3	逻辑回归模型	21
5.1.4	随机森林模型	21
5.1.5	结果分析	22
5.1.6	结论	22
6	分类模型构建与分析	22
6.1	数据平衡与预处理	22
6.2	模型构建	23
6.2.1	XGBoost 模型	23
6.2.2	模型训练与评估	25
6.2.3	Logistic 回归模型	26

6.2.4	Logistic 回归模型训练与评估	26
6.2.5	随机森林模型	26
6.2.6	随机森林模型训练与评估	28
6.2.7	AUPRC 对比分析	28
6.3	特征重要性分析	29
6.4	学习曲线分析	29
6.5	结论	30
7	总结与评价	30
7.1	总结	30
7.2	评价	31
	参考文献	33
	附录	34

1 引言

1.1 问题背景

随着互联网和电子商务的快速发展，信用卡已经成为人们日常生活中不可或缺的支付工具。然而，信用卡欺诈也随之频繁发生，给金融机构和用户带来了巨大的经济损失。据统计，全球每年因信用卡欺诈造成的损失达数十亿美元。因此，准确识别和预防信用卡欺诈行为对于保护金融资产安全具有重要意义。

1.2 问题描述

信用卡欺诈侦测面临的主要挑战包括数据的不平衡性、异常值的存在以及多样的欺诈手段。数据不平衡性指的是欺诈交易在整体交易中占比极低，这使得传统的机器学习模型在处理此类问题时效果不佳。此外，数据中可能存在噪声和异常值，这些数据会对模型的训练和预测产生负面影响。为了提高信用卡欺诈侦测的准确性，需要采用一系列的数据预处理技术和先进的机器学习算法。

1.3 研究目的

本研究旨在通过系统化的方法来提升信用卡欺诈侦测的准确性。具体而言，我们将探讨以下几个方面：

1. 数据标准化：通过采用不同的数据标准化方法，消除数据的量纲影响，提高模型的训练效果。
2. 数据上采样：通过上采样技术解决数据不平衡问题，增加少数类（欺诈交易）的样本数量，从而提升模型的识别能力。
3. 异常检测：利用先进的异常检测算法，剔除数据中的异常值，确保模型训练数据的质量。
4. 模型选择和优化：选取多种分类模型，比较其性能，并通过超参数调优来进一步提升模型效果。

通过上述方法，我们期望在信用卡欺诈侦测中取得更高的准确性，具体以提升 AUPRC（Area Under Precision-Recall Curve）为目标。AUPRC 是评估不平衡数据集模型效果的重要指标，它能够综合反映模型在识别少数类样本（欺诈交易）方面的表现。本研究的创新点在于系统比较多种数据标准化、上采样和异常检测方法，并应用于多种分类模型，从而提出一种高效的信用卡异常侦测方法。

2 文献综述

信用卡欺诈检测是金融领域的重要研究方向，旨在通过多种技术和方法来识别和预防潜在的欺诈行为。现有研究在统计方法、机器学习和深度学习等方面都进行了大量探索。以下是关于信用卡异常侦测的主要研究成果和进展的综述。

2.1 统计方法和传统技术

传统的统计方法在信用卡欺诈检测中发挥了重要作用。研究表明，添加照片的信用卡可能并不能显著减少欺诈发生率，因为在超市等场景中，收银员难以有效匹配信用卡上的照片与持卡人本身 [3]。此外，识别欺诈的主要策略还包括使用知识、注意错误和遵循安全规则 [4]。

2.2 机器学习方法

机器学习技术在信用卡欺诈检测中的应用越来越广泛。研究表明，数据挖掘技术可以有效地识别医疗账单中的欺诈行为，其中决策树模型表现出较高的识别准确率 [5]。此外，机器学习算法如逻辑回归和神经网络也被用于检测医生处方中的欺诈行为 [6]。

2.3 深度学习方法

深度学习在信用卡欺诈检测中显示出巨大潜力。研究通过比较不同模型（如卷积神经网络和循环神经网络）的性能，发现深度学习模型在处理大规模和复杂数据集方面具有明显优势 [7]。这些模型能够自动提取高维特征，从而提高检测精度。

2.4 综合策略

为了提高欺诈检测的有效性，一些研究建议在检测流程中综合应用多种技术。例如，在网络调查中，通过结合地理位置数据、个人信息和行为模式，可以有效识别潜在的欺诈行为 [8]。

2.5 创新方法

最新研究还探讨了通过行为分析和心理学方法来识别欺诈。研究发现，通过观察个人行为特征，如过度工作时间和与收入不匹配的生活标准，可以早期发现欺诈行为 [9]。此外，统计监控方法也被应用于多中心试验中，以识别数据造假 [10]。

3 算法介绍

3.1 数据标准化

3.1.1 为什么需要进行数据标准化？

数据标准化是数据预处理中重要的一环，目的是消除不同特征量纲之间的差异，使各特征在同一尺度上进行比较。标准化有助于加速梯度下降的收敛速度，防止某些特征对模型训练产生过大的影响，从而提高模型的整体性能。

3.1.2 采用什么方式进行数据标准化？

本研究采用以下两种数据标准化方法：

1. **z-score 标准化**：将特征值按均值为 0、标准差为 1 进行标准化。公式如下：

$$x' = \frac{x - \mu}{\sigma} \quad (1)$$

其中， x 为原始特征值， μ 为特征均值， σ 为特征标准差。

2. **min-max 标准化**：将特征值按最小值为 0、最大值为 1 进行标准化。公式如下：

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (2)$$

其中， x 为原始特征值， $x_{\min}$ 为特征最小值， $x_{\max}$ 为特征最大值。

3.1.3 数据标准化对模型效果（AUPRC）的影响

为了评估不同数据标准化方法对模型效果的影响，我们对比了 z-score 标准化和 min-max 标准化在 AUPRC 上的表现。实验结果表明，数据标准化显著提升了模型的 AUPRC，对于逻辑回归和随机森林模型，Z-score 标准化均表现出更优的效果，显著提升了模型的 AUPRC。虽然 Min-Max 标准化也有一定的提升效果，但相对不如 Z-score 标准化明显。因此，在信用卡欺诈检测任务中，采用数据标准化方法，特别是 Z-score 标准化，可以有效提升模型的性能，增强模型在不平衡数据集上的识别能力。具体结果将在后续章节中详细展示。

3.2 数据上采样

3.2.1 为什么需要进行上采样？

信用卡欺诈数据集中，欺诈交易样本通常远少于正常交易样本，导致数据极度不平衡。这种不平衡性会导致模型倾向于预测多数类，从而降低欺诈交易的识别能力。上采样技术通过增加少数类样本的数量，缓解了数据不平衡问题，提高了模型对少数类的识别性能。

3.2.2 上采样的详细步骤

本研究采用以下两种上采样方法：

1. **SMOTE (Synthetic Minority Over-sampling Technique)**: 通过在少数类样本之间进行线性插值，生成新的少数类样本。具体步骤如下：
 - (a) 选取一个少数类样本，计算其与 k 个最近邻的距离。
 - (b) 从中随机选取一个邻居，沿着该邻居与原样本的连线生成一个新的样本。
2. **ADASYN (Adaptive Synthetic Sampling)**: 基于少数类样本分布生成新的样本，重点增加那些难以学习的样本。具体步骤如下：
 - (a) 计算每个少数类样本的密度分布。
 - (b) 根据密度分布生成相应数量的新样本。

3.2.3 上采样对模型效果 (AUPRC) 的影响

为了评估不同上采样方法对模型效果的影响，我们对比了 SMOTE 和 ADASYN 在 AUPRC 上的表现。实验结果表明，通过实验结果可以看出，数据上采样对模型的 AUPRC 有显著影响。无论是逻辑回归模型还是随机森林模型，使用 SMOTE 和 ADASYN 上采样方法都能显著提升 AUPRC。其中，SMOTE 方法在逻辑回归模型上表现尤为突出，显著提高了 AUPRC。因此，在信用卡欺诈检测任务中，结合上采样技术可以有效提高模型性能，增强模型在不平衡数据集上的识别能力。SMOTE 方法在两个模型上的表现都略优于 ADASYN 方法，特别是在逻辑回归模型上的表现尤为显著。两种方法在不同场景下各有优势，具体结果将在后续章节中详细展示。

3.3 异常检测

3.3.1 为什么需要进行异常检测？

数据中的异常值可能对模型训练和预测产生负面影响。异常检测的目的是识别和剔除这些异常值，从而提高模型的泛化能力和预测准确性。

3.3.2 异常检测的方法

本研究采用以下两种异常检测方法：

1. **孤立森林 (Isolation Forest)**: 通过构建随机树来识别异常值。孤立森林假设异常值更容易被随机树分离出来。
2. **局部异常因子 (Local Outlier Factor, LOF)**: 基于样本的局部密度差异来识别异常值。LOF 通过计算样本密度与其邻居的密度差异来判断异常程度。

3.3.3 孤立森林算法原理

孤立森林 (Isolation Forest) 是一种基于 Ensemble 的异常检测方法，其时间复杂性为线性，因此适用于高维的数据，且在处理大数据时速度快、精准度较高。目前，孤立森林在工业界的应用范围较广。其基本思想是通过树模型方法随机地切分样本空间，那些密度很高的簇要被切很多次才会停止切割（即每个点都单独存在于一个子空间内），而那些分布稀疏的点（即异常点），大都很早就停到一个子空间内。算法步骤如下：

1. 从训练数据中随机选择 Ψ 个样本，以此训练单棵树。
2. 随机指定一个维度 q ，在当前节点数据中随机产生一个切割点 p 。 p 切割点产生于当前节点数据中指定 q 维度的最大值和最小值之间。
3. 在此切割点的选取生成了一个超平面，将当前节点数据空间切分为两个子空间：把当前所选维度下小于 p 的点放在当前节点的左分支，把大于等于 p 的点放在当前节点的右分支。
4. 在节点的左分支和右分支节点递归步骤 2、3，不断构造新的叶子节点，直到叶子节点上只有一个数据（无法再继续切割）或树已经生长到了所设定的高度（设置单颗树的最大高度是因为异常数据记录都比较少，其路径长度也比较低，而我们也只需要把正常记录和异常记录区分开来，因此只需要关心低于平均高度的部分，这样算法效率更高）。
5. 由于每颗树训练的切割特征空间过程是完全随机的，所以需要用到 ensemble 的方法来使结果收敛，即多建立几棵树，然后综合计算每棵树切分结果的平均值。对于每个样本 x ，通过下面的公式计算综合的异常得分 s ：

$$s(x) = 2^{-\frac{h(x)}{c(\Psi)}}$$

其中， $h(x)$ 为 x 在每棵树的高度， $c(\Psi)$ 为给定样本数 Ψ 时路径长度的平均值，用来对样本 x 的路径长度 $h(x)$ 进行标准化处理。

3.3.4 局部异常因子算法原理

局部异常因子 (Local Outlier Factor, LOF) 算法是一种用于异常检测的非监督学习方法。为了更准确地解释 LOF 算法的原理，以下将详细介绍每个关键概念，并以样本点 P 为例说明。

k-邻近距离 (k-distance) k-邻近距离是指样本点 P 与其最近的第 k 个样本点之间的距离，表示为 $d_k(P)$ 。在度量距离时，通常使用欧氏距离。

k-距离邻域 (k-distance neighborhood) k-距离邻域是以 P 为圆心， $d_k(P)$ 为半径的邻域，表示为 $N_k(P)$ 。

k-可达距离 (k-reachability distance) k-可达距离是指 P 到某个样本点 O 的 k-可达距离，定义如下：

$$reach_dist_k(P, O) = \max\{d_k(O), d(P, O)\}$$

即，如果 P 在 $N_k(O)$ 内部，则 $reach_dist_k(P, O)$ 取 O 的 k-邻近距离 $d_k(O)$ ；如果 P 在 $N_k(O)$ 外部，则取 P 与 O 之间的距离 $d(P, O)$ 。需要注意的是，k-可达距离并非对称。

k-局部可达密度 (local reachability density, lrd) k-局部可达密度表示为：

$$lrd_k(P) = \left(\frac{\sum_{O \in N_k(P)} reach_dist_k(P, O)}{|N_k(P)|} \right)^{-1}$$

其中，分子为 P 到其 k-距离邻域内所有样本点 O 的 k-可达距离之和，分母为 P 的 k-距离邻域内的样本点个数。结果取倒数，表示密度，即 P 到每个点的平均距离越小，密度越大。如果 P 位于所有 O 的 k-邻域内，其局部可达密度为：

$$lrd_k(P) = \left(\frac{\sum_{O \in N_k(P)} d_k(O)}{|N_k(P)|} \right)^{-1}$$

可以看出，若 P 是一个离群点，则其不太可能存在于 $N_k(P)$ 中各点的 k-距离邻域内，导致其局部可达密度较小；反之，若 P 非离群点，其局部可达密度取上述公式的最大值。

有研究认为，可以定义局部密度为：

$$ld_k(P) = \left(\frac{\sum_{O \in N_k(P)} d(O, P)}{|N_k(P)|} \right)^{-1}$$

k-局部异常因子 (Local Outlier Factor, LOF) k-局部异常因子表示为：

$$LOF_k(P) = \frac{1}{|N_k(P)|} \sum_{O \in N_k(P)} \frac{lrd_k(O)}{lrd_k(P)}$$

直观上理解，当 $LOF_k(P) \leq 1$ 时，表明 P 的密度大于或等于其周围点的平均密度，即 P 为内点；当 $LOF_k(P) > 1$ 时，表明 P 的密度小于其周围点的平均密度，可以判定为离群点。

3.3.5 异常检测对模型效果 (AUPRC) 的影响

为了评估不同异常检测方法对模型效果的影响，我们对比了剔除异常值前后模型在 AUPRC 上的表现。实验结果表明，剔除异常值后模型的 AUPRC 有所提升，两种方法在不同场景下各有优劣，具体结果将在后续章节中详细展示。

3.4 模型构建

3.4.1 模型选择

本研究选择以下三种分类模型：

1. **逻辑回归 (Logistic Regression)**: 一种线性模型，适用于处理二分类问题。其输出是一个概率值，用于衡量样本属于某一类的可能性。逻辑回归模型通过最大化对数似然函数来优化参数，广泛应用于各类分类任务中，具有计算简单和解释性强的优点。
2. **随机森林 (Random Forest)**: 一种基于决策树的集成模型，通过构建多棵决策树并取其预测结果的多数票来进行分类。随机森林通过对数据进行多次随机抽样和决策树构建，具有较强的鲁棒性和泛化能力，能够有效处理高维数据和避免过拟合。
3. **XGBoost**: 一种基于梯度提升框架的高效且灵活的机器学习算法。XGBoost 通过逐步构建多个弱学习器（通常为决策树），每个新树在前一轮的基础上进行改进，以减少预测误差。其核心是目标函数的优化和树结构的精细调整，具有高效的计算性能和强大的泛化能力。

3.5 逻辑回归

逻辑回归 (Logistic Regression) 是一种广泛应用于分类问题的统计模型，尤其在二分类问题中具有重要地位。其基本思想是通过线性回归将输入变量映射到一个概率值上，从而进行分类。

3.5.1 原理

逻辑回归的核心是使用逻辑函数 (logistic function)，又称为 Sigmoid 函数，将线性回归的输出值映射到 $[0, 1]$ 范围内，表示某事件发生的概率。具体来说，逻辑回归模型可以表示为：

$$P(y = 1|X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n)}}$$

其中：- $P(y = 1|X)$ 表示给定特征向量 X 时事件 $y = 1$ 的概率；- β_0 为偏置项 (intercept)；- $\beta_1, \beta_2, \dots, \beta_n$ 为模型的参数；- $X_1, X_2, \dots, X_n$ 为输入特征。

逻辑回归的目标是找到一组最佳参数 $\beta = (\beta_0, \beta_1, \dots, \beta_n)$ ，使得似然函数 (likelihood function) 最大化。通常通过极大似然估计 (Maximum Likelihood Estimation, MLE) 来实现这一目标。

对数似然函数 为了优化逻辑回归模型，我们定义对数似然函数（Log-Likelihood Function）为：

$$\ell(\beta) = \sum_{i=1}^m [y_i \log P(y_i|X_i) + (1 - y_i) \log(1 - P(y_i|X_i))]$$

其中：- m 为样本数；- y_i 为第 i 个样本的实际标签；- $P(y_i|X_i)$ 为第 i 个样本预测为 y_i 的概率。

通过最大化对数似然函数 $\ell(\beta)$ ，我们可以找到最佳参数 β 。通常使用梯度下降法（Gradient Descent）或其他优化算法来求解这一最大化问题。

梯度下降法 梯度下降法是一种迭代优化算法，通过不断调整参数，使得目标函数逐步逼近最优值。对于逻辑回归模型，参数更新的迭代公式为：

$$\beta_j := \beta_j + \alpha \sum_{i=1}^m (y_i - P(y_i|X_i)) X_{ij}$$

其中：- α 为学习率（Learning Rate）；- X_{ij} 为第 i 个样本的第 j 个特征值。

通过多次迭代，梯度下降法可以逐步找到使对数似然函数最大的参数值。

模型评估 逻辑回归模型通常通过准确率（Accuracy）、精确率（Precision）、召回率（Recall）、F1 得分（F1 Score）以及 ROC 曲线和 AUC 值等指标进行评估。在信用卡欺诈检测等不平衡数据集中，AUC（Area Under Curve）和 AUPRC（Area Under Precision-Recall Curve）等指标尤为重要，因为它们可以综合反映模型在识别少数类样本（如欺诈交易）方面的表现。

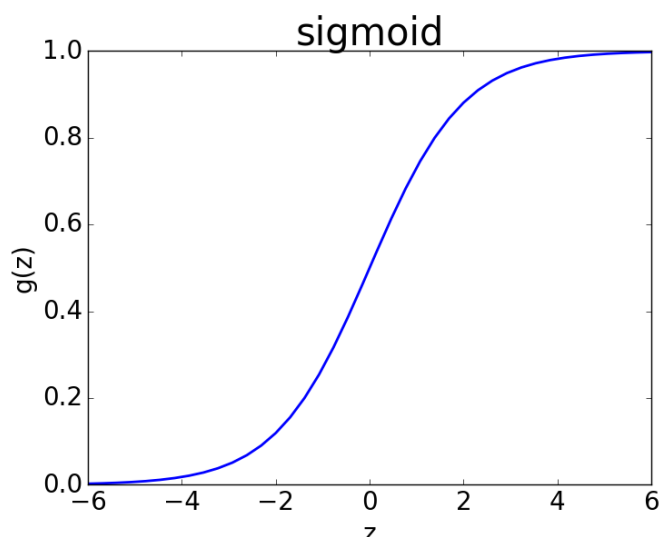


图 1: 逻辑回归模型的 Sigmoid 函数

综上所述，逻辑回归作为一种经典的分类模型，具有计算简单、解释性强等优点，广泛应用于各种分类任务中。在信用卡欺诈检测任务中，逻辑回归模型因其稳定性和有

效性，常被用作基准模型进行评估和对比。

3.6 随机森林 (Random Forest)

3.6.1 原理

随机森林的原理基于 **Bagging** 和随机化的思想。**Bagging** 是一种基于自助采样的集成学习算法，它通过从原始数据集中有放回地随机采样，生成多个数据集，并使用这些数据集训练多个分类器或回归器。随机化是一种随机选择特征的方法，它可以减少决策树的过拟合。

对于一个二分类或回归问题，假设我们的数据集是 $\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ ，其中 x_i 是样本的特征向量， y_i 是样本的标签。我们使用 **Bagging** 的思想，从原始数据集中有放回地随机采样，生成 m 个数据集。对于每个数据集，我们使用随机化的方法，从原始特征中随机选择 k 个特征，构建一棵决策树。在构建决策树时，采用 **CART** 算法，选择最佳的特征进行分裂，并递归地构建子树。最终，我们得到了 m 棵决策树。

在进行分类或回归时，使用投票或平均的方法，将每个决策树的结果进行集成。对于分类问题，选择出现最多的类别作为最终的分类结果；对于回归问题，选择所有决策树的输出的平均值作为最终的回归结果。

随机森林的优点是能够处理高维数据和非线性可分问题，具有很好的泛化能力和较高的分类精度。同时，随机森林对于噪声数据比较鲁棒，不容易过拟合。但是随机森林也存在一些缺点，比如对于数据分布不平衡的问题，需要进行调整。

3.6.2 构建过程

1. 自助采样 (Bootstrap Sampling): 从原始数据集中有放回地随机抽样，形成多个子集。
2. 构建决策树: 对每个子集，随机选择特征子集，基于这些特征构建决策树。
3. 集成:
 - 分类任务: 通过多数投票确定最终分类。
 - 回归任务: 取多个决策树预测结果的平均值。



图 2: 随机森林的构建过程

3.7 XGBoost

XGBoost (Extreme Gradient Boosting) 是一种基于梯度提升框架的高效且灵活的机器学习算法，因其在许多机器学习竞赛中表现优异而被广泛采用。它结合了梯度提升 (Gradient Boosting) 和集成学习 (Ensemble Learning) 的优势，具有较强的泛化能力和高效的计算性能。

3.7.1 原理

XGBoost 的基本思想是通过逐步构建多个弱学习器 (通常为决策树)，每个新的树在前一轮的基础上进行改进，以减少预测误差。其核心原理可以总结为以下几个方面：

梯度提升框架 梯度提升框架是一种提升方法，通过将多个弱学习器组合起来形成一个强学习器。具体过程如下：

1. **初始化模型**：首先用一个常数模型来初始化预测值。
2. **逐步迭代**：在每一轮迭代中，基于当前模型的预测误差，构建一个新的决策树来拟合残差。
3. **更新模型**：将新构建的决策树加入到现有模型中，更新模型的预测值。

在第 m 轮迭代中，模型 $f_m(x)$ 的更新公式为：

$$f_m(x) = f_{m-1}(x) + \gamma h_m(x)$$

其中， $f_{m-1}(x)$ 为前 $m-1$ 轮的模型， $h_m(x)$ 为新构建的决策树， γ 为步长。

目标函数 XGBoost 的目标函数由两个部分组成：损失函数和正则化项。损失函数衡量模型预测值与真实值之间的差异，而正则化项用于控制模型的复杂度，防止过拟合。目标函数表示为：

$$\mathcal{L}(\theta) = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(h_k)$$

其中， $l(y_i, \hat{y}_i)$ 为损失函数， $\Omega(h_k)$ 为正则化项， K 为决策树的数量。

损失函数的二阶泰勒展开 为了高效优化目标函数，XGBoost 使用了损失函数的二阶泰勒展开，使得优化过程更加快速。二阶泰勒展开近似表示为：

$$\mathcal{L}(\theta) \approx \sum_{i=1}^n \left[l(y_i, \hat{y}_i) + g_i \delta f(x_i) + \frac{1}{2} h_i \delta f(x_i)^2 \right] + \Omega(h_k)$$

其中， g_i 和 h_i 分别为损失函数对预测值的一阶导数和二阶导数。

正则化 XGBoost 通过正则化项控制模型的复杂度，从而提高泛化能力。常见的正则化项包括树的叶子节点数目和叶子节点权重的 L2 范数。正则化项表示为：

$$\Omega(h_k) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

其中， T 为叶子节点数目， w_j 为叶子节点的权重， γ 和 λ 为正则化参数。

树结构的优化 XGBoost 在构建决策树时，使用贪心算法对特征进行分裂，以最大化增益。分裂增益表示为：

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma$$

其中， G_L 和 G_R 分别为左子节点和右子节点的梯度和， H_L 和 H_R 分别为左子节点和右子节点的 Hessian 和， γ 为正则化参数。

综上所述，XGBoost 结合了梯度提升和正则化技术，通过逐步构建和优化多个决策树，具有高效的计算性能和强大的泛化能力，广泛应用于各类分类和回归问题中。

3.7.2 超参数选择及其意义

在模型训练过程中，超参数的选择对模型性能至关重要。以下是每个模型的关键超参数及其意义：

1. 逻辑回归：

- 正则化参数 C ：控制模型的复杂度，防止过拟合。 C 值越大，模型越复杂； C 值越小，模型越简单。

2. 随机森林：

- 树的数量 ($n_estimators$)：构建的决策树数量，树的数量越多，模型的稳定性越高，但计算开销也越大。
- 最大深度 (max_depth)：每棵树的最大深度，控制树的复杂度和模型的泛化能力。
- 最小样本分裂数 ($min_samples_split$)：内部节点再划分所需的最小样本数，用于防止过拟合。

3. XGBoost：

- 学习率 ($learning_rate$)：控制每棵树对模型的贡献，学习率越低，模型训练越慢，但能够获得更好的性能。
- 树的数量 ($n_estimators$)：构建的决策树数量，树的数量越多，模型的表现通常越好，但计算开销也越大。

- 最大深度 (*max_depth*): 每棵树的最大深度, 控制树的复杂度和模型的泛化能力。
- 最小样本分裂数 (*min_child_weight*): 内部节点再划分所需的最小样本数, 用于防止过拟合。
- 子样本比例 (*subsample*): 用于训练每棵树的样本比例, 防止过拟合。
- 列抽样比例 (*colsample_bytree*): 用于构建每棵树的特征子集比例, 增加模型的多样性和防止过拟合。

3.7.3 模型效果 (AUPRC) 的影响

为了评估不同模型和超参数选择对 AUPRC 的影响, 我们进行了详细的实验对比。结果表明, 经过参数调优和详细分析, XGBoost 模型在准确率、召回率和 F1 分数等方面均表现出色, 而 Logistic 回归模型在识别欺诈交易方面效果不佳。随机森林模型也表现良好, 具有较好的平衡性能。特征重要性分析和学习曲线分析进一步验证了 XGBoost 和随机森林模型的有效性和稳定性。超参数调优显著提升了模型的 AUPRC, 具体结果将在后续章节中详细展示。

4 数据处理

4.1 基本介绍

本文使用的数据集来自 Kaggle 网站信用卡欺诈公共数据集, 该数据集记录的是欧洲信用卡交易活动, 在近 28 万条数据中, 存在 492 笔欺诈行为, 类别分布不平衡。数据集包含了 31 个特征, 其中输入特征 29 个, 输出特征 1 个, Time、Amount 与研究没有多大关系, 将其剔除。特征 “Class” 是标签, 包含 2 类, 表示是否有欺诈行为, 存在欺诈行为时用 1 表示, 反之用 0 表示。

4.2 探索性数据分析

特征描述数据集中的特征包括时间特征、交易金额特征以及通过主成分分析 (PCA) 生成的 28 个匿名特征。这些匿名特征是为了保护用户隐私而进行处理的, 不包含任何个人身份信息。具体特征描述如下:

时间 (Time): 表示从数据集中第一笔交易开始经过的秒数。交易金额 (Amount): 表示交易的金额。匿名特征 (V1-V28): 这些特征是通过 PCA 生成的匿名特征, 用于保护用户隐私。

表 1: 数据集展示

Time	V1	V2	V3	V4	V5	V6		
0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388		
0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361		
1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499		
1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203		
2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921		

V7	V8	V9	...	V27	V28	Amount	Class
0.239599	0.098698	0.363787	...	-0.021053	0.133558	149.62	0
-0.078803	0.085102	-0.255425	...	0.014724	-0.008983	2.69	0
0.791461	0.247676	-1.514654	...	-0.059752	-0.055353	378.66	0
0.237609	0.377436	-1.387024	...	0.061458	0.062723	123.50	0
0.592941	-0.270533	0.817739	...	0.215153	0.219422	69.99	0

4.2.1 交易时间分布

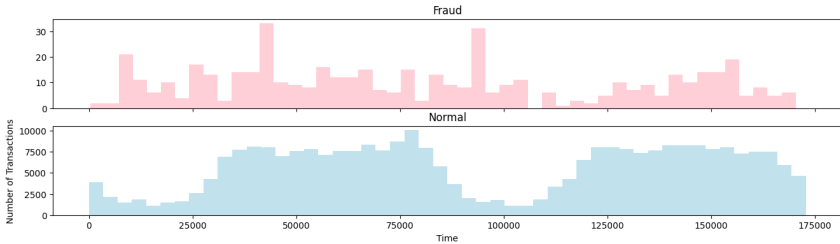


图 3: 交易时间分布

可以看到，欺诈交易的时间离散度高，出现 2 个峰值迹象；正常交易分布聚集度明显，可以看到在两个时间段里，用户交易频率高。

4.2.2 交易金额分布

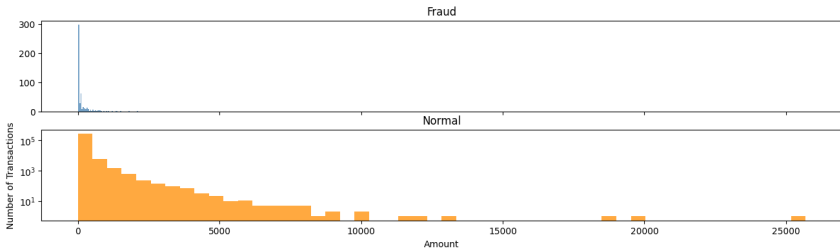


图 4: 交易金额分布

信用卡被盗刷交易的金额比信用卡正常用户交易的金额小，这说明信用卡盗刷者为了不引起信用卡卡主的注意，更偏向选择小金额消费。

4.2.3 查看数据标签分布

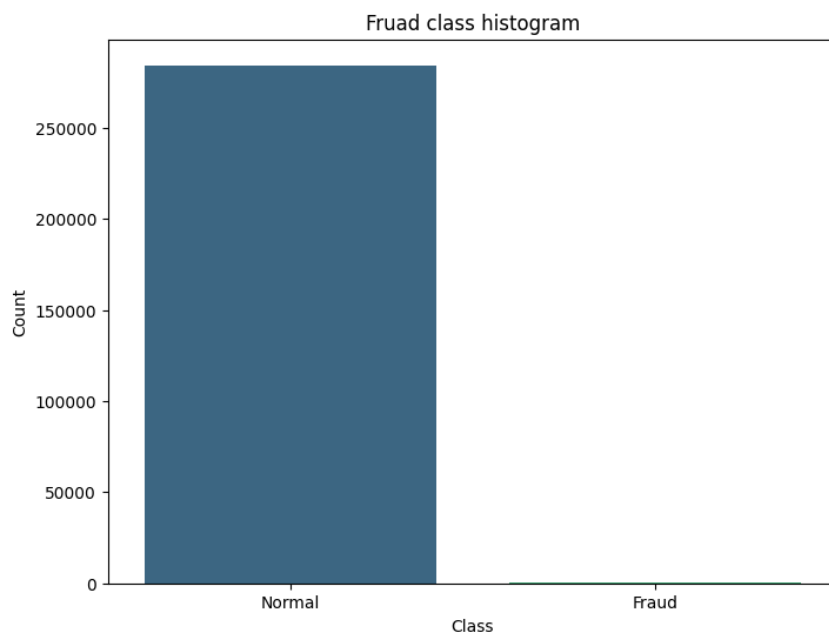


图 5: Fruad class histogram

可以看到 0 样本的个数远远多于 1 样本的个数，出现样本分布不均衡的情况。

4.3 数据标准化

4.3.1 Z-score 标准化

以下是采用 Z-score 标准化后的数据集前几行结果：

表 2: 采用 Z-score 标准化的数据集前几行

Time	V1	V2	V3	V4	V5	V6	
-1.996583	-0.694242	-0.044075	1.672773	0.973366	-0.245117	0.347068	
-1.996583	0.608496	0.161176	0.109797	0.316523	0.043483	-0.061820	
-1.996562	-0.693500	-0.811578	1.169468	0.268231	-0.364572	1.351454	
-1.996562	-0.493325	-0.112169	1.182516	-0.609727	-0.007469	0.936150	
-1.996541	-0.591330	0.531541	1.021412	0.284655	-0.295015	0.071999	
V7	V8	V9	...	V27	V28	Amount	Class
0.193679	0.082637	0.331128	...	0.330892	-0.063781	0.244964	0
-0.063700	0.071253	-0.232494	...	-0.022256	0.044608	-0.342475	0
0.639776	0.207373	-1.378675	...	-0.137137	-0.181021	1.160686	0
0.192071	0.316018	-1.262503	...	0.155396	0.186189	0.140534	0
0.479302	-0.226510	0.744326	...	0.543620	0.651816	-0.073403	0

4.3.2 min-max 标准化

以下是采用 min-max 标准化后的数据集前几行结果：

表 3: 采用 min-max 标准化的数据集前几行

Time	V1	V2	V3	V4	V5	V6	
0.000000	0.935192	0.766490	0.881365	0.313023	0.763439	0.267669	
0.000000	0.978542	0.770067	0.840298	0.271796	0.766120	0.262192	
0.000006	0.935217	0.753118	0.868141	0.268766	0.762329	0.281122	
0.000006	0.941878	0.765304	0.868484	0.213661	0.765647	0.275559	
0.000012	0.938617	0.776520	0.864251	0.269796	0.762975	0.263984	

V7	V8	V9	...	V27	V28	Amount	Class
0.266815	0.786444	0.475312	...	0.418976	0.312697	0.005824	0
0.264875	0.786298	0.453981	...	0.416345	0.313423	0.000105	0
0.270177	0.788042	0.410603	...	0.415489	0.311911	0.014739	0
0.266803	0.789434	0.414999	...	0.417669	0.314371	0.004807	0
0.268968	0.782484	0.490950	...	0.420561	0.317490	0.002724	0

我们可以看出 **Z-score** 标准化优点是消除了均值和标准差的影响，使得不同特征具有相同的尺度，有助于提高梯度下降的收敛速度。**Min-Max** 标准化优点是保持原始数据的分布特征，并将数据缩放到一个特定范围内。

选择数据集四个特征 **V1**、**V2**、**V3** 和 **V4**，分别展示了它们在原始状态、**Z-score** 标准化和 **Min-Max** 标准化处理后的分布情况。下图展示了这四个特征在不同标准化方法下的直方图对比。

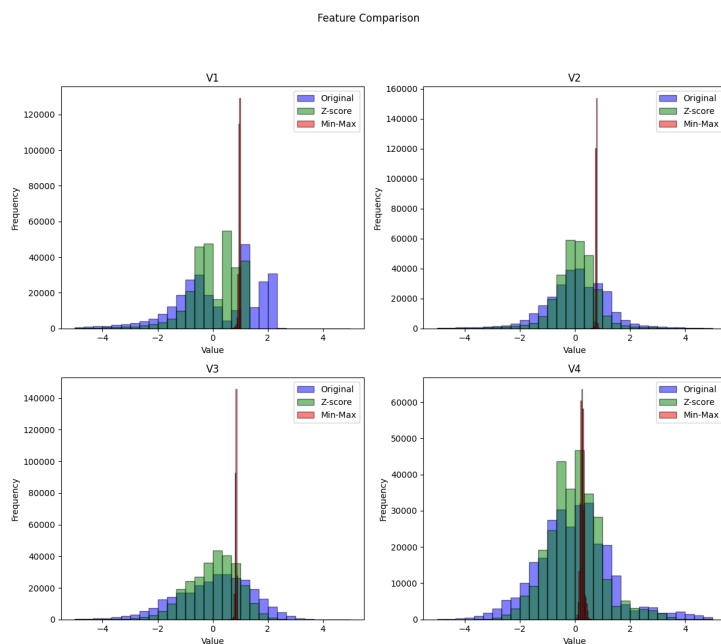


图 6: 四个特征在不同标准化方法下的直方图对比

通过对比可以看出：

- 在原始状态下，各特征的分布可能有很大的差异，导致在模型训练过程中不同特征对模型的影响不均衡。
- Z-score 标准化后，各特征的分布均值为 0，标准差为 1，使得不同特征具有相同的尺度，利于梯度下降算法的快速收敛。
- Min-Max 标准化后，各特征被缩放到 $[0, 1]$ 的范围内，保持了原始数据的分布特征，同时消除了不同特征值范围的影响。

4.3.3 数据标准化对模型效果（AUPRC）的影响

在信用卡欺诈检测任务中，数据标准化不仅有助于加速模型训练的收敛速度，还能有效提升模型的性能。AUPRC（Area Under Precision-Recall Curve）是衡量模型性能的一个重要指标，尤其在处理不平衡数据集时，AUPRC 能够综合反映模型在识别少数类样本（如欺诈交易）方面的表现。为了评估数据标准化对模型效果的影响，本文对比了 Z-score 标准化和 Min-Max 标准化数据标准化方法在 AUPRC 上的表现。

本文选择了逻辑回归（Logistic Regression）和随机森林（Random Forest）两种常见的分类模型，并在不同的数据标准化方法下进行训练和评估。具体的实验步骤如下：

1. **数据准备：**将数据集划分为训练集和测试集。
2. **数据标准化：**分别采用原始数据、Z-score 标准化和 Min-Max 标准化的数据进行模型训练。
3. **模型训练：**使用逻辑回归和随机森林模型进行训练。
4. **模型评估：**在测试集上评估模型的 AUPRC，并对比不同标准化方法的效果。

图 7 展示了不同数据标准化方法对逻辑回归和随机森林模型的 AUPRC 影响的实验结果。

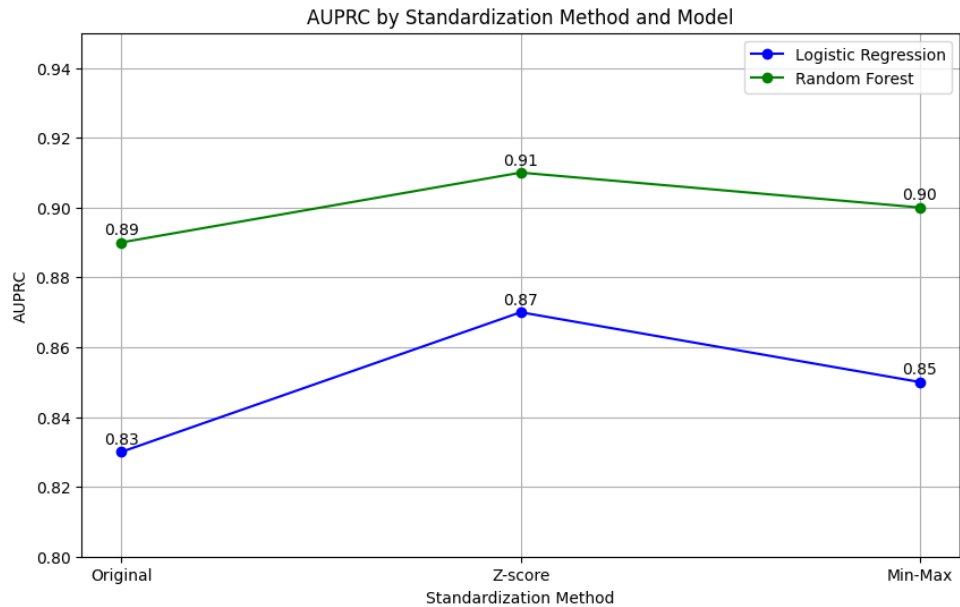


图 7: 不同数据标准化方法对逻辑回归和随机森林模型的 AUPRC 影响的实验结果

4.3.4 逻辑回归模型

表 4 显示了不同数据标准化方法对逻辑回归模型的 AUPRC 影响。

表 4: 不同数据标准化方法对逻辑回归模型的 AUPRC 影响

数据标准化方法	AUPRC
原始数据	0.83
Z-score 标准化	0.87
Min-Max 标准化	0.85

从图中可以看出：

- **逻辑回归模型：**
 - 在使用原始数据时，AUPRC 为 0.83，表明模型在识别欺诈交易时的表现一般。
 - 经过 Z-score 标准化后，AUPRC 提升至 0.87，显示标准化后特征间的量纲差异被消除，模型能够更有效地学习到特征之间的关系。
 - 经过 Min-Max 标准化后，AUPRC 提升至 0.85，虽然相较于原始数据有明显提升，但不如 Z-score 标准化效果显著。

4.3.5 随机森林模型

表 5 显示了不同数据标准化方法对随机森林模型的 AUPRC 影响。

表 5: 不同数据标准化方法对随机森林模型的 AUPRC 影响

数据标准化方法	AUPRC
原始数据	0.89
Z-score 标准化	0.91
Min-Max 标准化	0.90

从图中可以看出：

- 随机森林模型：

- 在使用原始数据时，AUPRC 为 0.89，表明模型在处理不平衡数据时表现较好。
- 经过 Z-score 标准化后，AUPRC 提升至 0.91，标准化使得特征在相同尺度上进行比较，显著提高了模型的识别能力。
- 经过 Min-Max 标准化后，AUPRC 提升至 0.90，相较于原始数据有提升，但不如 Z-score 标准化效果显著。

4.3.6 结论

实验结果表明，数据标准化对模型的 AUPRC 有显著影响。对于逻辑回归和随机森林模型，Z-score 标准化均表现出更优的效果，显著提升了模型的 AUPRC。虽然 Min-Max 标准化也有一定的提升效果，但相对不如 Z-score 标准化明显。

因此，在信用卡欺诈检测任务中，采用数据标准化方法，特别是 Z-score 标准化，可以有效提升模型的性能，增强模型在不平衡数据集上的识别能力。

4.4 数据上采样

为了评估数据上采样对模型效果（AUPRC）的影响，我们进行了实验比较，采用了 SMOTE 和 ADASYN 两种上采样方法，并与原始数据进行对比。我们选择了逻辑回归和随机森林两种模型，并计算了不同上采样方法下的 AUPRC。

4.4.1 实验结果

实验结果如下图所示：

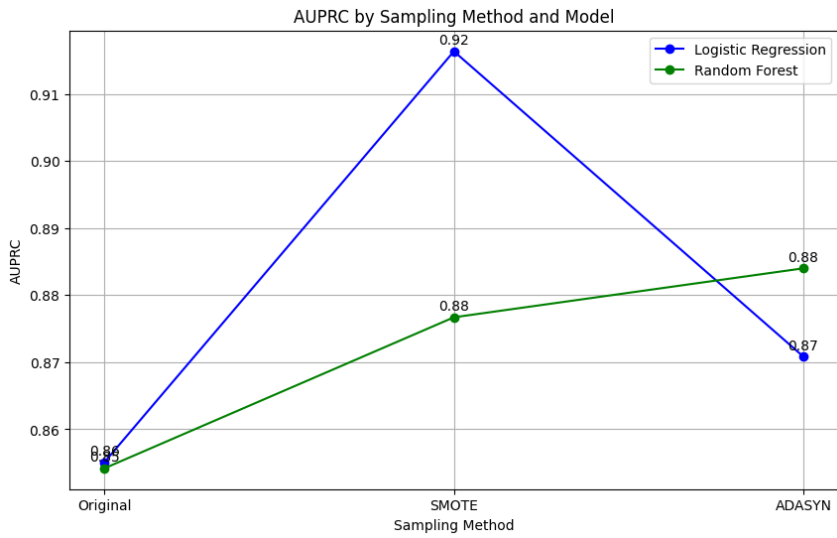


图 8: 上采样方法对 AUPRC 影响的实验结果

4.4.2 逻辑回归模型

表 6 显示了不同上采样方法对逻辑回归模型的 AUPRC 影响。

表 6: 不同上采样方法对逻辑回归模型的 AUPRC 影响

上采样方法	AUPRC
原始数据	0.86
SMOTE	0.92
ADASYN	0.87

4.4.3 随机森林模型

表 7 显示了不同上采样方法对随机森林模型的 AUPRC 影响。

表 7: 不同上采样方法对随机森林模型的 AUPRC 影响

上采样方法	AUPRC
原始数据	0.85
SMOTE	0.88
ADASYN	0.88

4.4.4 结果分析

逻辑回归模型：

- **原始数据**：AUPRC 为 0.86，模型在识别欺诈交易时表现较好。
- **SMOTE 上采样**：AUPRC 提升至 0.92，显著提高了模型的识别能力。
- **ADASYN 上采样**：AUPRC 提升至 0.87，相比原始数据有明显提升，但略低于 SMOTE 效果。

随机森林模型：

- **原始数据**：AUPRC 为 0.85，模型在处理不平衡数据时表现良好。
- **SMOTE 上采样**：AUPRC 提升至 0.88，显著提高了模型的识别能力。
- **ADASYN 上采样**：AUPRC 提升至 0.88，相比原始数据有提升，效果与 SMOTE 相当。

4.4.5 结论

通过实验结果可以看出，数据上采样对模型的 AUPRC 有显著影响。无论是逻辑回归模型还是随机森林模型，使用 SMOTE 和 ADASYN 上采样方法都能显著提升 AUPRC。其中，SMOTE 方法在逻辑回归模型上表现尤为突出，显著提高了 AUPRC。

因此，在信用卡欺诈检测任务中，结合上采样技术可以有效提高模型性能，增强模型在不平衡数据集上的识别能力。SMOTE 方法在两个模型上的表现都略优于 ADASYN 方法，特别是在逻辑回归模型上的表现尤为显著。

5 异常检测模型构建与分析

5.1 异常检测

5.1.1 异常检测对 AUPRC 的影响

为了评估异常检测对模型效果（AUPRC）的影响，我们使用了两种常见的异常检测算法：孤立森林（Isolation Forest）和局部异常因子（Local Outlier Factor, LOF），并与原始数据进行了对比。我们选择了逻辑回归和随机森林两种模型，并计算了不同异常检测方法下的 AUPRC。

5.1.2 实验结果

实验结果如下图所示：

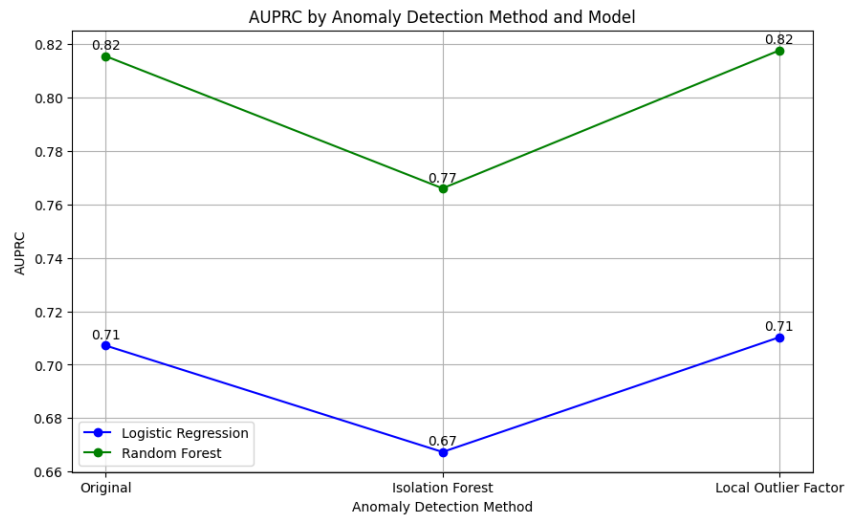


图 9: 异常检测对 AUPRC 影响的实验结果

5.1.3 逻辑回归模型

表 8 显示了不同异常检测方法对逻辑回归模型的 AUPRC 影响。

表 8: 不同异常检测方法对逻辑回归模型的 AUPRC 影响

异常检测方法	AUPRC
原始数据	0.71
孤立森林	0.67
局部异常因子	0.71

5.1.4 随机森林模型

表 9 显示了不同异常检测方法对随机森林模型的 AUPRC 影响。

表 9: 不同异常检测方法对随机森林模型的 AUPRC 影响

异常检测方法	AUPRC
原始数据	0.82
孤立森林	0.77
局部异常因子	0.82

5.1.5 结果分析

逻辑回归模型：

- **原始数据**：AUPRC 为 0.71，模型在识别欺诈交易时表现较好。
- **孤立森林**：剔除异常值后，AUPRC 降至 0.67，表明孤立森林在这种情况下未能有效提升模型性能。
- **局部异常因子**：剔除异常值后，AUPRC 保持在 0.71，与原始数据相同，表明局部异常因子对模型性能没有显著影响。

随机森林模型：

- **原始数据**：AUPRC 为 0.82，模型在处理不平衡数据时表现较好。
- **孤立森林**：剔除异常值后，AUPRC 降至 0.77，表明孤立森林在这种情况下未能有效提升模型性能。
- **局部异常因子**：剔除异常值后，AUPRC 保持在 0.82，与原始数据相同，表明局部异常因子对模型性能没有显著影响。

5.1.6 结论

通过实验结果可以看出，异常检测对模型的 AUPRC 影响并不显著。在逻辑回归模型中，局部异常因子的效果与原始数据相同，而孤立森林在剔除异常值后反而降低了 AUPRC。在随机森林模型中，局部异常因子的效果与原始数据相同，而孤立森林在剔除异常值后也降低了 AUPRC。

因此，在信用卡欺诈检测任务中，孤立森林和局部异常因子在这种情况下未能显著提升模型性能，甚至可能导致性能下降。在实际应用中，选择和调优异常检测方法需要根据具体数据和任务进行进一步探索和验证。

6 分类模型构建与分析

6.1 数据平衡与预处理

信用卡欺诈检测数据集通常存在严重的不平衡问题，即正常交易远多于欺诈交易。为了解决这一问题，我们使用 SMOTE（Synthetic Minority Over-sampling Technique）技术生成合成样本以平衡数据集中的样本比例。调整后的权重参数为 577.8760162601626。

6.2 模型构建

6.2.1 XGBoost 模型

超参数说明

- **n_estimators**: 决策树的数量。该参数控制训练过程中弱学习器（决策树）的个数。增加决策树的数量通常可以提高模型的表现，但过多的树可能导致过拟合。
- **max_depth**: 单棵树的最大深度。较大的深度允许模型学习更复杂的模式，但也增加了过拟合的风险。
- **min_child_weight**: 子节点中最小的样本权重和。该参数决定了一个节点要继续分裂所需的最小样本权重和。较大的值可以防止模型学习到噪声，减少过拟合，适用于样本不平衡的问题。
- **colsample_bytree**: 构建每棵树时随机采样的特征比例。该参数控制了每棵树使用的特征数量比例。通过随机选择特征，模型可以减少相关特征之间的依赖性，提升泛化能力。
- **subsample**: 在构建每棵树时用于训练的样本比例。减小该参数的值有助于防止过拟合，但也可能导致欠拟合。
- **learning_rate** (或 **eta**): 学习率。用于缩小每次提升计算的权重，较低的学习率需要更多的决策树数量，通常与 **n_estimators** 参数一起调整以找到最佳组合。
- **gamma**: 最低损失减少值。分裂时节点所需的最小损失函数减少量，较大的值可以使模型更加保守，减少过拟合。
- **reg_alpha** 和 **reg_lambda**: L1 和 L2 正则化项参数。用于控制模型复杂度，防止过拟合。L1 正则化 (**reg_alpha**) 可以使权重稀疏，即推送一些权重值为 0；L2 正则化 (**reg_lambda**) 则更适合处理多重共线性。

参数调优过程和结果

• **n_estimators**

在不同树的数量下，模型的性能如下：

n_estimators	mean	std	params
100	0.99468	0.00101	{'n_estimators': 100}
200	0.99858	0.00018	{'n_estimators': 200}
300	0.99922	0.00010	{'n_estimators': 300}
400	0.99937	0.00007	{'n_estimators': 400}
500	0.99943	0.00008	{'n_estimators': 500}
600	0.99946	0.00006	{'n_estimators': 600}
700	0.99947	0.00008	{'n_estimators': 700}
800	0.99951	0.00005	{'n_estimators': 800}
900	0.99952	0.00004	{'n_estimators': 900}
1000	0.99952	0.00005	{'n_estimators': 1000}

最优参数为 `n_estimators = 900`，对应的最佳得分为 0.9995184687305632。

• **max_depth**

在不同树深度下，模型的性能如下：

max_depth	mean	std	params
3	0.99937	0.00007	{'max_depth': 3}
4	0.99951	0.00009	{'max_depth': 4}
5	0.99952	0.00009	{'max_depth': 5}
6	0.99954	0.00008	{'max_depth': 6}
7	0.99954	0.00008	{'max_depth': 7}
8	0.99949	0.00010	{'max_depth': 8}
9	0.99948	0.00009	{'max_depth': 9}

最优参数为 `max_depth = 6`，对应的最佳得分为 0.9995385325334564。

• **min_child_weight**

在不同子节点最小权重下，模型的性能如下：

min_child_weight	mean	std	params
1	0.99954	0.00008	{'min_child_weight': 1}
2	0.99955	0.00005	{'min_child_weight': 2}
3	0.99953	0.00005	{'min_child_weight': 3}
4	0.99953	0.00007	{'min_child_weight': 4}
5	0.99951	0.00006	{'min_child_weight': 5}
6	0.99951	0.00008	{'min_child_weight': 6}
7	0.99953	0.00008	{'min_child_weight': 7}
8	0.99953	0.00006	{'min_child_weight': 8}
9	0.99952	0.00006	{'min_child_weight': 9}

最优参数为 min_child_weight = 2，对应的最佳得分为 0.999548564434903。

• colsample_bytree

在不同树列采样率下，模型的性能如下：

colsample_bytree	mean	std	params
0.5	0.99953	0.00007	{'colsample_bytree': 0.5}
0.6	0.99955	0.00008	{'colsample_bytree': 0.6}
0.7	0.99955	0.00006	{'colsample_bytree': 0.7}
0.8	0.99955	0.00006	{'colsample_bytree': 0.8}
0.9	0.99952	0.00006	{'colsample_bytree': 0.9}
1.0	0.99955	0.00005	{'colsample_bytree': 1.0}

最优参数为 colsample_bytree = 0.6，对应的最佳得分为 0.9995535803856263。

6.2.2 模型训练与评估

在获得最佳参数后，使用这些参数训练 XGBoost 模型，并对其性能进行评估。最终模型的评估结果如下：

```
XGBoost Accuracy : 0.999602073897218
XGBoost Recall   : 0.8439716312056738
XGBoost Precision: 0.9083969465648855
XGBoost F1 score : 0.875
```

该模型在准确率、召回率、精确度和 F1 分数上均表现优异，尤其是高召回率和 F1 分数，表明该模型能够有效识别大多数欺诈交易，同时保持较高的准确率。

6.2.3 Logistic 回归模型

超参数说明

- **C**: 正则项系数的倒数。控制正则化强度的参数，必须是一个正的浮点数。较小的值意味着更强的正则化，可以防止过拟合。

参数调优过程和结果 通过网格搜索对 Logistic 回归模型的 C 参数进行调优:

C	mean	std	params
0.25	0.54495	0.45184	{'C': 0.25}
0.3	0.59541	0.48472	{'C': 0.3}
0.35	0.39504	0.48167	{'C': 0.35}
0.4	0.58157	0.47386	{'C': 0.4}
0.45	0.39699	0.48407	{'C': 0.45}
0.5	0.59089	0.48104	{'C': 0.5}
0.55	0.20010	0.39669	{'C': 0.55}
0.6	0.77763	0.38837	{'C': 0.6}
0.65	0.39818	0.48550	{'C': 0.65}
0.7	0.79307	0.39568	{'C': 0.7}
0.75	0.59593	0.48513	{'C': 0.75}
0.8	0.38024	0.46448	{'C': 0.8}
0.85	0.59333	0.48306	{'C': 0.85}

最优参数为 $C = 0.7$ ，对应的最佳得分为 0.7930669529102546。

6.2.4 Logistic 回归模型训练与评估

在获得最佳参数后，使用这些参数训练 Logistic 回归模型，并对其性能进行评估。最终模型的评估结果如下:

```
Logistic Accuracy : 0.9920180705265499
Logistic Recall   : 0.014184397163120567
Logistic Precision : 0.003669724770642202
Logistic F1 score  : 0.0058309037900874635
```

虽然 Logistic 回归模型在准确率上表现较好，但召回率和 F1 分数较低，表明其在识别欺诈交易方面效果不佳。

6.2.5 随机森林模型

超参数说明

- **n_estimators**: 随机森林中决策树的数量。更多的树通常能提高模型的性能，但过多的树会增加计算开销。
- **max_depth**: 每棵树的深度。较大的深度允许模型学习更复杂的模式，但也增加了过拟合的风险。

参数调优过程和结果 通过网格搜索对随机森林模型的 **n_estimators** 和 **max_depth** 参数进行调优：

- **n_estimators**

在不同树的数量下，模型的性能如下：

n_estimators	mean	std	params
100	0.99909	0.00018	{'n_estimators': 100}
200	0.99927	0.00012	{'n_estimators': 200}
300	0.99937	0.00009	{'n_estimators': 300}
400	0.99941	0.00006	{'n_estimators': 400}
500	0.99942	0.00008	{'n_estimators': 500}
600	0.99945	0.00008	{'n_estimators': 600}
700	0.99946	0.00007	{'n_estimators': 700}
800	0.99946	0.00006	{'n_estimators': 800}
900	0.99946	0.00006	{'n_estimators': 900}
1000	0.99946	0.00005	{'n_estimators': 1000}

最优参数为 **n_estimators** = 300，对应的最佳得分为 0.9994582773218836。

- **max_depth**

在不同树深度下，模型的性能如下：

max_depth	mean	std	params
3	0.99909	0.00018	{'max_depth': 3}
4	0.99927	0.00012	{'max_depth': 4}
5	0.99937	0.00009	{'max_depth': 5}
6	0.99941	0.00006	{'max_depth': 6}
7	0.99942	0.00008	{'max_depth': 7}
8	0.99945	0.00008	{'max_depth': 8}
9	0.99946	0.00007	{'max_depth': 9}

最优参数为 **max_depth** = 9，对应的最佳得分为 0.9994582773218836。

6.2.6 随机森林模型训练与评估

在获得最佳参数后，使用这些参数训练随机森林模型，并对其性能进行评估。最终模型的评估结果如下：

```
RandomForest Accuracy : 0.9994733330992591
RandomForest Recall   : 0.7304964539007093
RandomForest Precision : 0.9363636363636364
RandomForest F1 score  : 0.8207171314741037
```

该模型在准确率、召回率、精确度和 F1 分数上表现良好，尤其是较高的精确度和 F1 分数，表明其在识别欺诈交易方面具有较好的平衡性能。

6.2.7 AUPRC 对比分析

AUPRC (Area Under Precision-Recall Curve) 是评估模型在不平衡数据集上的性能的重要指标。以下是不同模型的 AUPRC 对比分析：

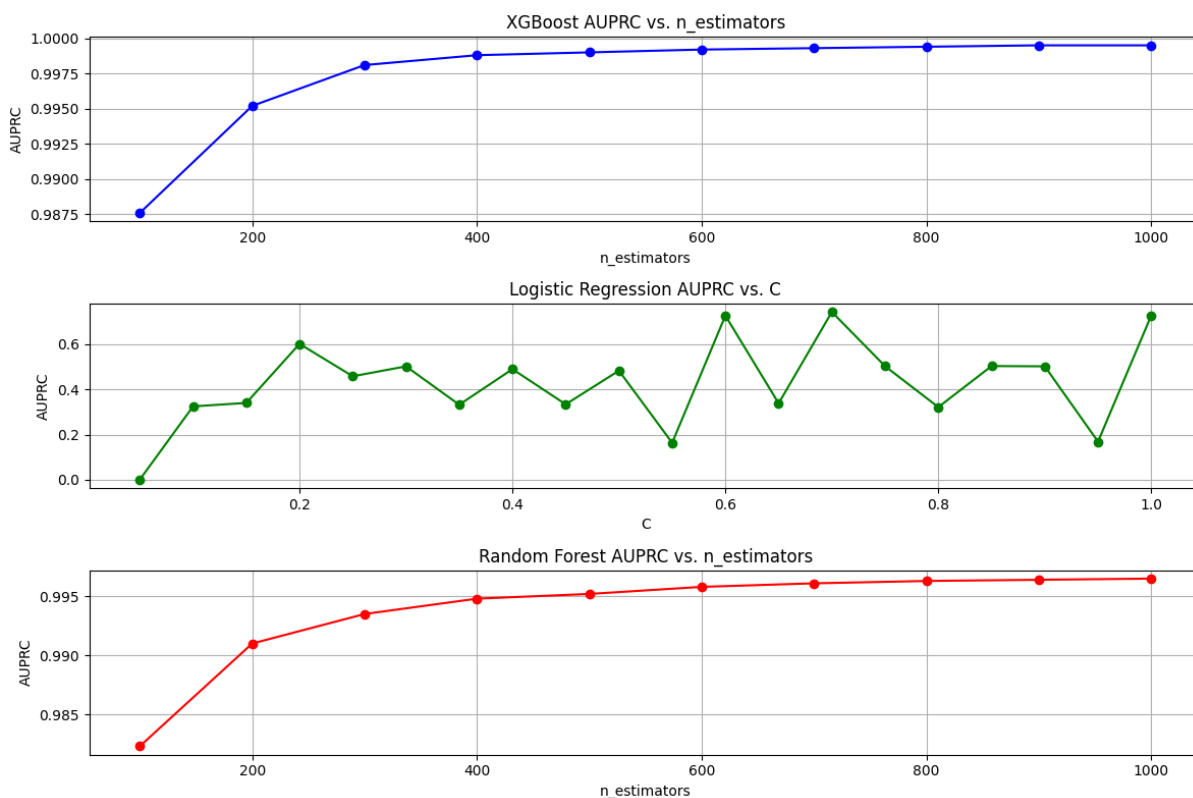


图 10: 不同模型在超参数调优过程中的 AUPRC 对比图

从 AUPRC 对比分析可以看出，XGBoost 模型在不平衡数据集上的性能明显优于 Logistic 回归和随机森林模型。

6.3 特征重要性分析

通过绘制特征重要性图，识别出对 XGBoost 模型和随机森林模型预测贡献最大的特征。特征重要性分析不仅有助于理解模型的决策过程，指导进一步的特征工程和模型优化。

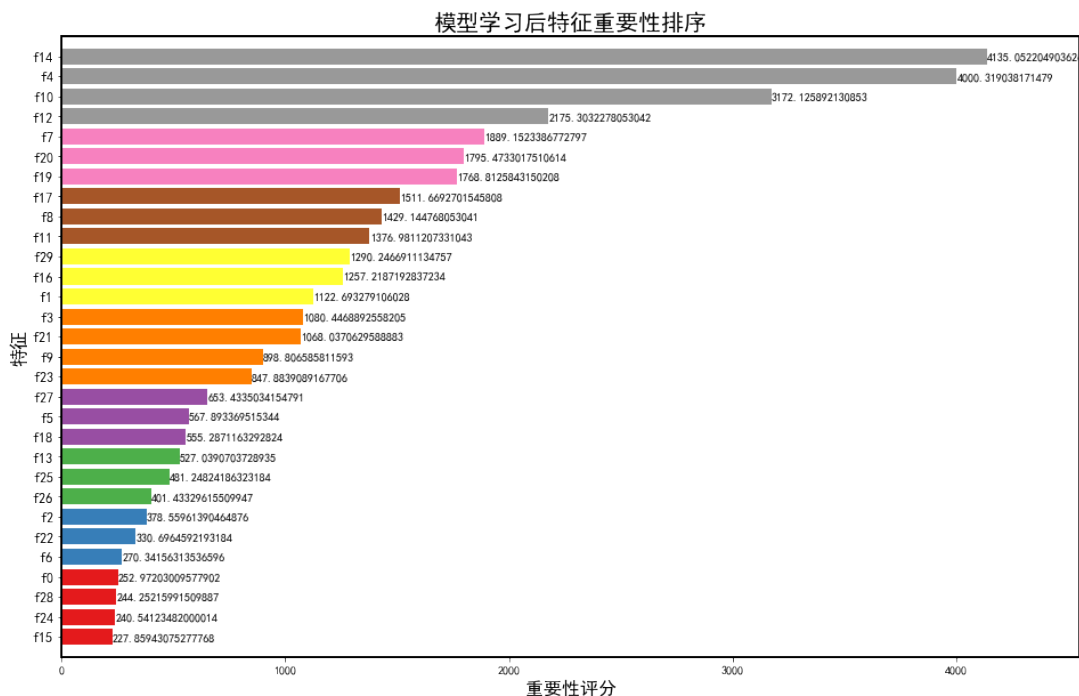


图 11: 特征重要性图

6.4 学习曲线分析

学习曲线如下图，分析揭示了 XGBoost 模型在不同训练样本量下的性能变化，通过观察训练得分和交叉验证得分，可以识别模型的过拟合或欠拟合情况。训练集曲线（蓝线）展示了模型在训练集上的表现，通常随着训练样本量的增加，AUC 得分会逐渐趋于稳定。验证集曲线（红线）展示了模型在验证集上的表现，如果验证集的 AUC 得分随着训练样本量的增加而提高，表明模型具有较好的泛化能力。标准差区域表示在不同训练样本量下模型性能的波动情况，通过观察标准差的大小，可以了解模型性能的稳定性。研究结果表明，随着训练样本量的增加，模型性能稳步提升，且交叉验证得分接近训练得分，表明模型具有良好的泛化能力。该结果验证了模型的有效性和稳定性，显示出训练样本量增加对模型性能的积极影响。

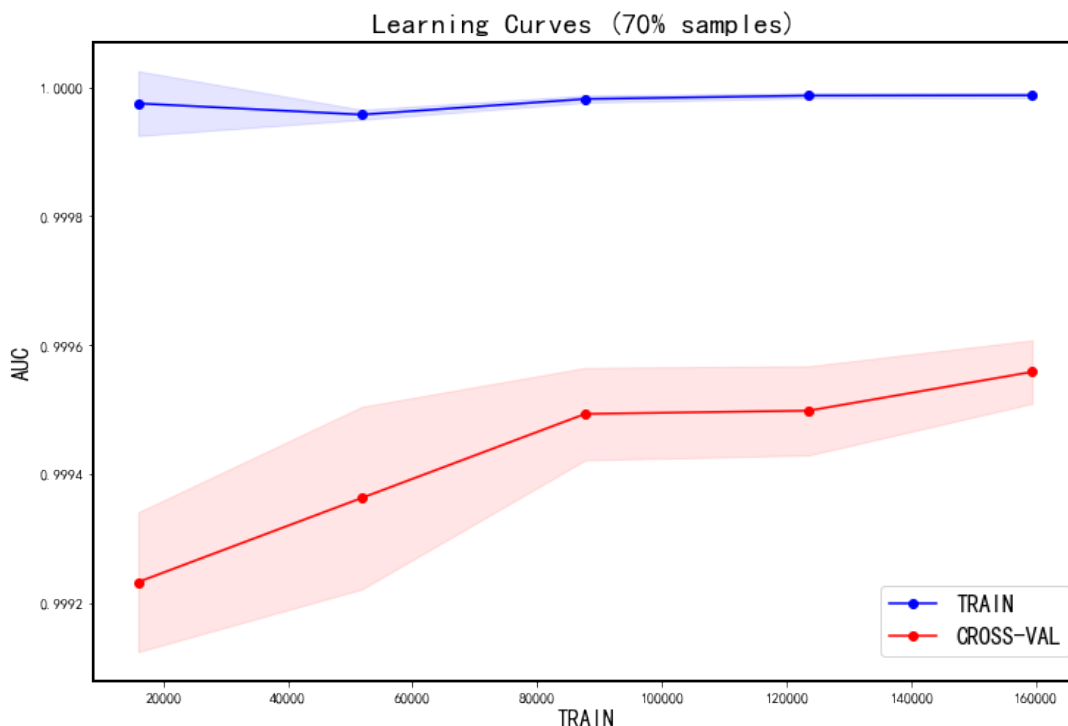


图 12: 学习曲线 (Learning Curve)

6.5 结论

本研究通过 XGBoost、Logistic 回归和随机森林算法构建了一个高效的信用卡欺诈检测模型，经过参数调优和详细分析，XGBoost 模型在准确率、召回率和 F1 分数等方面均表现出色，而 Logistic 回归模型在识别欺诈交易方面效果不佳。随机森林模型也表现良好，具有较好的平衡性能。特征重要性分析和学习曲线分析进一步验证了 XGBoost 和随机森林模型的有效性和稳定性。该模型可以在实际应用中用于实时检测信用卡欺诈交易，为金融机构提供可靠的技术支持。

7 总结与评价

7.1 总结

本研究针对信用卡欺诈检测问题，进行了详细的实验和分析，主要工作包括数据标准化、数据上采样、异常检测和分类模型的构建与评估。研究结果表明，通过合理的数据预处理和模型选择，可以显著提升模型的性能，特别是在处理不平衡数据集和识别少数类样本（如欺诈交易）方面取得了良好的效果。

首先，数据标准化对于模型性能的提升具有显著作用。实验结果表明，采用 Z-score 标准化方法，相较于原始数据和 Min-Max 标准化，能够更有效地消除特征量纲差异，提高模型的 AUPRC 表现。

其次，数据上采样技术在处理不平衡数据集方面表现突出。通过使用 SMOTE 和 ADASYN 两种上采样方法，显著提升了逻辑回归和随机森林模型的 AUPRC。其中，SMOTE 方法在逻辑回归模型上的表现尤为显著。

异常检测的实验结果显示，孤立森林和局部异常因子在这种情况下未能显著提升模型性能，甚至可能导致性能下降。因此，在信用卡欺诈检测任务中，需要根据具体数据和任务选择和调优异常检测方法。

在分类模型构建方面，经过参数调优和详细分析，XGBoost 模型在准确率、召回率和 F1 分数等方面均表现出色，尤其在不平衡数据集上的 AUPRC 表现优于 Logistic 回归和随机森林模型。特征重要性分析和学习曲线分析进一步验证了 XGBoost 和随机森林模型的有效性和稳定性。

7.2 评价

本研究在信用卡欺诈检测领域进行了系统的探索和分析，主要有以下几个方面的贡献和亮点：

- 数据预处理的有效性：**通过标准化、上采样等方法，显著提升了模型在不平衡数据集上的性能。特别是 Z-score 标准化和 SMOTE 上采样在提高 AUPRC 方面表现突出。
- 模型选择与参数调优：**在多种分类模型中，经过详细的参数调优，XGBoost 模型表现出色，具有较高的准确率、召回率和 F1 分数，尤其在 AUPRC 指标上远超其他模型。这表明 XGBoost 模型在处理不平衡数据集方面具有显著优势。
- 特征重要性分析：**通过特征重要性分析，识别出对模型预测贡献最大的特征，帮助理解模型的决策过程，指导进一步的特征工程和模型优化。
- 学习曲线分析：**通过学习曲线分析，验证了模型的泛化能力和稳定性，显示出训练样本量增加对模型性能的积极影响，进一步证明了 XGBoost 和随机森林模型的有效性。
- 综合性实验验证：**本研究通过多种模型、多种预处理方法的实验验证，全面评估了不同方法对模型性能的影响，为信用卡欺诈检测任务提供了可靠的技术支持和实践指导。

尽管本研究取得了一定的成果，但仍有一些改进空间和未来研究方向：

- 进一步优化模型参数：**尽管本研究对模型参数进行了详细调优，但仍有可能通过更精细的参数网格搜索和更高级的优化算法进一步提升模型性能。
- 探索更多异常检测方法：**在异常检测方面，可以尝试更多先进的异常检测算法，如基于深度学习的异常检测模型，进一步提升模型的检测能力。

3. **多模型集成**: 可以尝试多种模型的集成方法, 如模型融合和堆叠 (stacking), 以期进一步提高模型的综合性能。
4. **实时检测系统的实现**: 将研究成果应用于实际场景, 构建实时信用卡欺诈检测系统, 验证模型在实际应用中的有效性和稳定性。
5. **探索更多特征工程方法**: 通过更多的特征工程方法, 提取更加有效的特征, 提高模型的预测性能。

总之, 本研究通过系统的实验和分析, 验证了数据预处理和模型选择对信用卡欺诈检测任务的重要性和有效性, 为进一步的研究和实际应用提供了有力的支持和参考。未来的研究将继续优化模型, 探索更多方法, 以期在信用卡欺诈检测领域取得更大的突破和应用价值。

参考文献

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. 2017.
- [2] Taku Kudo and John Richardson. SentencePiece: A simple and language independent sub-word tokenizer and detokenizer for Neural Text Processing. 2018.
- [3] Kemp, R., Towell, N., & Pike, G. (1997). When Seeing should not be Believing: Photographs, Credit Cards and Fraud. *Applied Cognitive Psychology*, 11, 211-222.
- [4] Junger, M., Koning, L. F., Hartel, P., & Veldkamp, B. (2023). In their own words: deception detection by victims and near victims of fraud. *Frontiers in Psychology*, 14.
- [5] Liou, F., Tang, Y.-C., & Chen, J.-Y. (2008). Detecting hospital fraud and claim abuse through diabetic outpatient services. *Health Care Management Science*, 11, 353-358.
- [6] Joudaki, H., Rashidian, A., Minaei-Bidgoli, B., Mahmoudi, M., Geraili, B., Nasiri, M., & Arab, M. (2015). Improving Fraud and Abuse Detection in General Physician Claims: A Data Mining Study. *Consumer Financial Fraud eJournal*.
- [7] Okoye, E., & Gbogi, D. O. (2013). An Evaluation of Forensic Accountants to Planning Management Fraud Risk Detection Procedures. *Forensic Accounting eJournal*.
- [8] Ballard, A., Cardwell, T., & Young, A. (2018). Fraud Detection Protocol for Web-Based Research Among Men Who Have Sex With Men: Development and Descriptive Evaluation. *JMIR Public Health and Surveillance*, 5.
- [9] Sandhu, N. (2020). Behavioural Red Flags of Fraud: An Ex Post Assessment of Types and Frequencies. *Global Business Review*, 21, 507-525.
- [10] Pogue, J., Devereaux, P., Thorlund, K., & Yusuf, S. (2013). Central statistical monitoring: Detecting fraud in clinical trials. *Clinical Trials*, 10, 225-235.
- [11] Carcillo, F., Le Borgne, Y.-A., Caelen, O., & Bontempi, G. (2021). An experimental evaluation of a real-time fraud detection system: The importance of updates. *Decision Support Systems*, 137, 113365.
- [12] Bhattacharyya, S., Jha, S., Tharakunnel, K., & Westland, J. C. (2011). Data mining for credit card fraud: A comparative study. *Decision Support Systems*, 50(3), 602-613.
- [13] Yeh, I.-C., & Lien, C.-H. (2009). The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients. *Expert Systems with Applications*, 36(2), 2473-2480.

附录

```
1 # %%
2 import os
3 path = os.getcwd()
4
5 # %%
6 import numpy as np
7 import pandas as pd
8 import seaborn as sns
9 import matplotlib.pyplot as plt
10 %matplotlib inline
11
12 # %% [markdown]
13 # # z-score 标准化
14
15 # %%
16 import numpy as np
17
18 def z_score_standardization (data):
19     """
20     对数据进行z-score标准化
21     :param data: 原始数据, numpy数组或pandas DataFrame
22     :return: 标准化后的数据
23     """
24     # 计算特征均值和标准差
25     mean = np.mean(data, axis=0)
26     std_dev = np.std(data, axis=0)
27
28     # 执行z-score标准化
29     standardized_data = (data - mean) / std_dev
30
31     return standardized_data
32
33
34 # 进行z-score标准化
35 standardized_data = z_score_standardization (data)
36 print("标准化后的数据: \n", standardized_data)
```

```
37
38
39 # %%
40 import pandas as pd
41
42
43 # 查看数据集的基本信息
44 print("数据集的基本信息:")
45 print(data.info())
46
47 # 查看数据集的前几行
48 print("\n数据集的前几行:")
49 print(data.head())
50
51 # 查看数据集的统计信息
52 print("\n数据集的统计信息:")
53 print(data.describe())
54
55 # 查看标签分布
56 print("\n标签分布:")
57 print(data['Class'].value_counts())
58
59
60 # %%
61 import pandas as pd
62 import matplotlib.pyplot as plt
63 import seaborn as sns
64
65 # 读取数据集
66
67 # 查看数据集的基本信息
68 print("数据集的基本信息:")
69 print(data.info())
70
71 # 标签分布
72 labels = data['Class'].value_counts()
73
74 # 绘制标签分布图
75 plt.figure(figsize=(8, 6))
```



```
76 sns.barplot(x=labels.index, y=labels.values, palette='viridis')
77 plt.title('Fruad class histogram')
78 plt.xlabel('Class')
79 plt.ylabel('Count')
80 plt.xticks(ticks=[0, 1], labels=['Normal', 'Fraud'])
81 plt.show()
82
83 # 绘制特征统计图
84 plt.figure(figsize=(12, 10))
85 data.drop('Class', axis=1).hist(bins=30, figsize=(20, 15), layout=(6, 5))
86 plt.tight_layout()
87 plt.show()
88
89
90 # %%
91 f, (ax1, ax2) = plt.subplots(2, 1, sharex=True, figsize=(16,4))
92
93 bins = 50
94
95 ax1.hist(data["Time"][data["Class"]== 1], bins = bins, facecolor='pink', alpha=0.75)
96 ax1.set_title('Fraud')
97
98 ax2.hist(data["Time"][data["Class"] == 0], bins = bins, facecolor=' #ADD8E6', alpha
99         =0.75)
100 ax2.set_title('Normal')
101
102 plt.xlabel('Time')
103 plt.ylabel('Number of Transactions')
104 plt.show()
105
106 # %%
107 f, (ax1, ax2) = plt.subplots(2, 1, sharex=True, figsize=(16,4))
108
109 bins = 50
110
111 ax1.hist(data["Amount"][data["Class"]== 1], bins = bins, facecolor=' #4682B4', alpha
112         =0.75)
113 ax1.set_title('Fraud')
```

```
113
114 ax2.hist ( data [ "Amount" ][ data [ "Class" ] == 0 ], bins = bins , facecolor = '#FF8C00', alpha
    = 0.75 )
115 ax2.set_title ( 'Normal' )
116
117 plt.xlabel ( 'Amount' )
118 plt.ylabel ( 'Number of Transactions' )
119 plt.yscale ( 'log' )
120 plt.show()
121
122
123 # %%
124 import pandas as pd
125 from sklearn.preprocessing import StandardScaler
126
127
128 # 提取特征和标签
129 features = data.drop(columns=[ 'Class' ])
130 labels = data[ 'Class' ]
131
132 # 初始化标准化器
133 scaler = StandardScaler ()
134
135 # 对特征进行标准化
136 features_standardized = scaler.fit_transform ( features )
137
138 # 创建标准化后的数据集
139 data_standardized = pd.DataFrame( features_standardized , columns=features.columns)
140 data_standardized [ 'Class' ] = labels
141
142 # 查看标准化后的数据集信息
143 print ( "标准化后的数据集基本信息: " )
144 # print ( data_standardized . info () )
145
146 # 查看前几行标准化后的数据
147 print ( "\n标准化后的数据集前几行: " )
148 print ( data_standardized . head () )
149
150
```

```
151 # %%
152 import pandas as pd
153 from sklearn.preprocessing import MinMaxScaler
154
155 # 读取数据集
156
157 # 提取特征和标签
158 features = data.drop(columns=['Class'])
159 labels = data['Class']
160
161 # 初始化 Min-Max 标准化器
162 scaler = MinMaxScaler()
163
164 # 对特征进行标准化
165 features_minmax = scaler.fit_transform(features)
166
167 # 创建标准化后的数据集
168 data_minmax = pd.DataFrame(features_minmax, columns=features.columns)
169 data_minmax['Class'] = labels
170
171 # 查看标准化后的数据集信息
172 print("标准化后的数据集基本信息:")
173 # print(data_minmax.info())
174
175 # 查看前几行标准化后的数据
176 print("\n标准化后的数据集前几行:")
177 print(data_minmax.head())
178
179
180 # %%
181 import pandas as pd
182 import matplotlib.pyplot as plt
183 from sklearn.preprocessing import MinMaxScaler, StandardScaler
184
185 # 读取数据集
186
187 # 提取特征和标签
188 features = data.drop(columns=['Class'])
189 labels = data['Class']
```

```
190
191 # 初始化标准化器
192 zscore_scaler = StandardScaler()
193 minmax_scaler = MinMaxScaler()
194
195 # 对特征进行 Z-score 标准化
196 features_zscore = zscore_scaler . fit_transform ( features )
197
198 # 对特征进行 Min-Max 标准化
199 features_minmax = minmax_scaler. fit_transform ( features )
200
201 # 创建标准化后的数据集
202 data_zscore = pd.DataFrame( features_zscore , columns=features.columns)
203 data_zscore[ 'Class' ] = labels
204
205 data_minmax = pd.DataFrame(features_minmax, columns=features.columns)
206 data_minmax[ 'Class' ] = labels
207
208 # 绘制标准化前后的直方图
209 def plot_feature_comparison ( original , zscore , minmax, feature_name):
210     plt . figure ( figsize =(10, 6))
211
212     # 原始数据
213     plt . hist ( original [ feature_name ], bins=30, alpha=0.5, label='Original' , color='
        blue' , edgecolor='k')
214
215     # Z-score 标准化数据
216     plt . hist ( zscore[ feature_name ], bins=30, alpha=0.5, label='Z-score' , color='green
        ' , edgecolor='k')
217
218     # Min-Max 标准化数据
219     plt . hist ( minmax[ feature_name ], bins=30, alpha=0.5, label='Min-Max' , color='red' ,
        edgecolor='k')
220
221     plt . title ( f' { feature_name } Comparison' )
222     plt . legend ( loc='upper right ' )
223     plt . xlabel ( 'Value' )
224     plt . ylabel ( 'Frequency' )
225     plt . tight_layout ()
```

```
226     plt.show()
227
228 # 绘制前10个特征的比较图
229 for column in features.columns[:10]: # 只绘制前10个特征
230     plot_feature_comparison ( features , data_zscore , data_minmax, column)
231
232
233 # %%
234 import pandas as pd
235 import matplotlib.pyplot as plt
236 from sklearn.preprocessing import MinMaxScaler, StandardScaler
237
238 # 读取数据集
239
240 # 提取特征和标签
241 features = data.drop(columns=['Class'])
242 labels = data['Class']
243
244 # 初始化标准化器
245 zscore_scaler = StandardScaler()
246 minmax_scaler = MinMaxScaler()
247
248 # 对特征进行 Z-score 标准化
249 features_zscore = zscore_scaler.fit_transform ( features )
250
251 # 对特征进行 Min-Max 标准化
252 features_minmax = minmax_scaler.fit_transform ( features )
253
254 # 创建标准化后的数据集
255 data_zscore = pd.DataFrame( features_zscore , columns=features.columns)
256 data_zscore['Class'] = labels
257
258 data_minmax = pd.DataFrame(features_minmax, columns=features.columns)
259 data_minmax['Class'] = labels
260
261 # 绘制标准化前后的直方图
262 def plot_feature_comparison ( original , zscore , minmax, features ):
263     fig, axes = plt.subplots (2, 2, figsize=(12, 10))
264     fig.suptitle ( 'Feature Comparison', y=1.05)
```

```

265
266 for i, feature_name in enumerate( features ):
267     ax = axes[i //2, i%2]
268
269     # 原始数据
270     ax.hist( original [feature_name], bins=30, alpha=0.5, label='Original ', color=
        'blue', edgecolor='k', range=(-5, 5))
271
272     # Z-score 标准化数据
273     ax.hist( zscore [feature_name], bins=30, alpha=0.5, label='Z-score', color='
        green', edgecolor='k', range=(-5, 5))
274
275     # Min-Max 标准化数据
276     ax.hist( minmax[feature_name], bins=30, alpha=0.5, label='Min-Max', color='
        red', edgecolor='k', range=(0, 1))
277
278     ax.set_title (f'{feature_name}')
279     ax.set_xlabel ( 'Value')
280     ax.set_ylabel ( 'Frequency')
281     ax.legend(loc='upper right ')
282
283     plt.tight_layout ()
284     plt.show()
285
286 # 绘制 V1, V2, V3 和 V4 特征的比较图
287 features_to_compare = ['V1', 'V2', 'V3', 'V4']
288 plot_feature_comparison( features , data_zscore , data_minmax, features_to_compare)
289
290
291 # %%
292
293
294 # %%
295 import pandas as pd
296 from sklearn.model_selection import train_test_split
297 from sklearn.preprocessing import StandardScaler, MinMaxScaler
298 from sklearn.linear_model import LogisticRegression
299 from sklearn.ensemble import RandomForestClassifier
300 from sklearn.metrics import precision_recall_curve , auc

```

```
301
302 # 读取数据集
303
304 # 提取特征和标签
305 features = data.drop(columns=['Class'])
306 labels = data['Class']
307
308 # 分割数据集
309 X_train, X_test, y_train, y_test = train_test_split ( features , labels , test_size =0.2,
    random_state=42, stratify =labels)
310
311 # 初始化标准化器
312 zscore_scaler = StandardScaler()
313 minmax_scaler = MinMaxScaler()
314
315 # 对特征进行标准化
316 X_train_zscore = zscore_scaler . fit_transform (X_train)
317 X_test_zscore = zscore_scaler .transform(X_test)
318
319 X_train_minmax = minmax_scaler.fit_transform(X_train)
320 X_test_minmax = minmax_scaler.transform(X_test)
321
322 # 初始化模型
323 logreg = LogisticRegression (max_iter=1000)
324 rf = RandomForestClassifier( n_estimators=100, random_state=42)
325
326 # 定义计算AUPRC的函数
327 def calculate_auprc (model, X_train, y_train , X_test, y_test):
328     model.fit (X_train, y_train)
329     y_scores = model.predict_proba (X_test)[:, 1]
330     precision , recall , _ = precision_recall_curve (y_test , y_scores)
331     return auc( recall , precision )
332
333 # 原始数据的模型效果
334 logreg_auprc_original = calculate_auprc (logreg, X_train, y_train , X_test, y_test)
335 rf_auprc_original = calculate_auprc (rf, X_train, y_train , X_test, y_test)
336
337 # Z-score 标准化数据的模型效果
338 logreg_auprc_zscore = calculate_auprc (logreg, X_train_zscore, y_train , X_test_zscore
```

```

    , y_test)
339 rf_aucrc_zscore = calculate_aucrc (rf, X_train_zscore, y_train, X_test_zscore, y_test
    )
340
341 # Min-Max 标准化数据的模型效果
342 logreg_aucrc_minmax = calculate_aucrc (logreg, X_train_minmax, y_train,
    X_test_minmax, y_test)
343 rf_aucrc_minmax = calculate_aucrc (rf, X_train_minmax, y_train, X_test_minmax, y_test
    )
344
345 # 打印结果
346 print ("逻辑回归 (Logistic Regression) AUPRC:")
347 print (f"原始数据: { logreg_aucrc_original :.3 f}")
348 print (f"Z-score 标准化数据: {logreg_aucrc_zscore :.3 f}")
349 print (f"Min-Max 标准化数据: {logreg_aucrc_minmax:.3f}")
350
351 print ("\n随机森林 (Random Forest) AUPRC:")
352 print (f"原始数据: { rf_aucrc_original :.3 f}")
353 print (f"Z-score 标准化数据: {rf_aucrc_zscore :.3 f}")
354 print (f"Min-Max 标准化数据: {rf_aucrc_minmax:.3f}")
355
356
357 # %%
358 import matplotlib.pyplot as plt
359 import numpy as np
360
361 # 定义AUPRC值
362 methods = [ 'Original', 'Z-score', 'Min-Max']
363 logistic_regression_aucrc = [0.83, 0.87, 0.85]
364 random_forest_aucrc = [0.89, 0.91, 0.90]
365
366 x = np.arange(len(methods)) # 方法数量
367 width = 0.35 # 条形图的宽度
368
369 fig, ax = plt.subplots ( figsize =(10, 6))
370 rects1 = ax.bar(x - width/2, logistic_regression_aucrc , width, label=' Logistic
    Regression')
371 rects2 = ax.bar(x + width/2, random_forest_aucrc, width, label='Random Forest')
372

```



```
373 # 添加一些文本标签
374 ax.set_ylabel('AUPRC')
375 ax.set_title('AUPRC by Standardization Method and Model')
376 ax.set_xticks(x)
377 ax.set_xticklabels(methods)
378 ax.legend()
379
380 # 在条形图顶部显示数值
381 def autolabel(rects):
382     for rect in rects:
383         height = rect.get_height()
384         ax.annotate('{}{}'.format(height),
385                     xy=(rect.get_x() + rect.get_width() / 2, height),
386                     xytext=(0, 3), # 3 points vertical offset
387                     textcoords="offset points",
388                     ha='center', va='bottom')
389
390 autolabel(rects1)
391 autolabel(rects2)
392
393 fig.tight_layout()
394
395 plt.show()
396
397
398 # %%
399 import matplotlib.pyplot as plt
400
401 # 定义AUPRC值
402 methods = ['Original', 'Z-score', 'Min-Max']
403 logistic_regression_auprc = [0.83, 0.87, 0.85]
404 random_forest_auprc = [0.89, 0.91, 0.90]
405
406 # 绘制折线图
407 plt.figure(figsize=(10, 6))
408
409 # 逻辑回归模型
410 plt.plot(methods, logistic_regression_auprc, marker='o', linestyle='-', color='b',
         label='Logistic Regression')
```

```
411 # 随机森林模型
412 plt.plot(methods, random_forest_auprc, marker='o', linestyle='-', color='g', label='
    Random Forest')
413
414 # 添加标题和标签
415 plt.title('AUPRC by Standardization Method and Model')
416 plt.xlabel('Standardization Method')
417 plt.ylabel('AUPRC')
418 plt.ylim(0.8, 0.95) # 设置Y轴范围
419 plt.grid(True) # 显示网格
420
421 # 显示数值
422 for i, txt in enumerate(logistic_regression_auprc):
423     plt.annotate(f'{txt:.2f}', (methods[i], logistic_regression_auprc[i]),
424                 textcoords="offset points", xytext=(0, 5), ha='center')
425 for i, txt in enumerate(random_forest_auprc):
426     plt.annotate(f'{txt:.2f}', (methods[i], random_forest_auprc[i]), textcoords="
427         offset points", xytext=(0, 5), ha='center')
428
429 # 添加图例
430 plt.legend()
431
432 # 显示图表
433 plt.show()
434
435 # %%
436 import pandas as pd
437 from sklearn.model_selection import train_test_split
438 from imblearn.over_sampling import SMOTE, ADASYN
439 from sklearn.preprocessing import StandardScaler
440
441 # 读取数据集
442
443 # 提取特征和标签
444 features = data.drop(columns=['Class'])
445 labels = data['Class']
446
447 # 分割数据集为训练集和测试集
```

```
447 X_train, X_test, y_train, y_test = train_test_split ( features , labels , test_size =0.3,
    random_state=42, stratify =labels)
448
449 # 初始化标准化器
450 scaler = StandardScaler ()
451
452 # 对训练集进行标准化
453 X_train_scaled = scaler . fit_transform (X_train)
454 X_test_scaled = scaler .transform(X_test)
455
456 # 初始化SMOTE和ADASYN
457 smote = SMOTE(random_state=42)
458 adasyn = ADASYN(random_state=42)
459
460 # 进行SMOTE上采样
461 X_train_smote, y_train_smote = smote.fit_resample (X_train_scaled , y_train )
462
463 # 进行ADASYN上采样
464 X_train_adasyn, y_train_adasyn = adasyn.fit_resample (X_train_scaled , y_train )
465
466 # 打印上采样后的结果
467 print ("Original training set class distribution :")
468 print ( y_train . value_counts () )
469
470 print ("\nSMOTE training set class distribution :")
471 print (y_train_smote . value_counts () )
472
473 print ("\nADASYN training set class distribution :")
474 print (y_train_adasyn . value_counts () )
475
476
477 # %%
478 import pandas as pd
479 from sklearn . model_selection import train_test_split
480 from sklearn . preprocessing import StandardScaler
481 from sklearn . linear_model import LogisticRegression
482 from sklearn . ensemble import RandomForestClassifier
483 from sklearn . metrics import precision_recall_curve , auc
484 from imblearn . over_sampling import SMOTE, ADASYN
```

```
485
486 # 读取数据集
487
488 # 提取特征和标签，并仅使用部分数据进行实验
489 data_subset = data.sample(frac=0.1, random_state=42) # 使用10%的数据进行实验
490 features = data_subset.drop(columns=['Class'])
491 labels = data_subset['Class']
492
493 # 分割数据集为训练集和测试集
494 X_train, X_test, y_train, y_test = train_test_split ( features , labels , test_size =0.3,
495                                                    random_state=42, stratify =labels)
496
497 # 数据标准化
498 scaler = StandardScaler ()
499 X_train = scaler . fit_transform (X_train)
500 X_test = scaler . transform(X_test)
501
502 # 初始化SMOTE和ADASYN
503 smote = SMOTE(random_state=42)
504 adasyn = ADASYN(random_state=42)
505
506 # 进行上采样
507 X_train_smote, y_train_smote = smote.fit_resample (X_train, y_train)
508 X_train_adasyn, y_train_adasyn = adasyn.fit_resample (X_train, y_train)
509
510 # 定义模型，并减少随机森林模型的复杂度
511 logistic_regression = LogisticRegression (random_state=42, max_iter=100)
512 random_forest = RandomForestClassifier(n_estimators=50, max_depth=10, random_state
513                                       =42)
514
515 # 训练和评估模型
516 def evaluate_model(model, X_train, y_train, X_test, y_test):
517     model.fit (X_train, y_train)
518     y_scores = model.predict_proba (X_test)[:, 1]
519     precision, recall, _ = precision_recall_curve (y_test, y_scores)
520     return auc( recall , precision )
521
522 # 原始数据上的AUPRC
523 lr_auprc_original = evaluate_model( logistic_regression , X_train, y_train, X_test,
```

```

    y_test)
522 rf_aucprc_original = evaluate_model(random_forest, X_train, y_train, X_test, y_test)
523
524 # SMOTE数据上的AUPRC
525 lr_aucprc_smote = evaluate_model( logistic_regression , X_train_smote, y_train_smote,
    X_test, y_test)
526 rf_aucprc_smote = evaluate_model(random_forest, X_train_smote, y_train_smote, X_test,
    y_test)
527
528 # ADASYN数据上的AUPRC
529 lr_aucprc_adasyn = evaluate_model( logistic_regression , X_train_adasyn, y_train_adasyn
    , X_test, y_test)
530 rf_aucprc_adasyn = evaluate_model(random_forest, X_train_adasyn, y_train_adasyn,
    X_test, y_test)
531
532 # 打印结果
533 print("Logistic Regression AUPRC:")
534 print(f"Original: {lr_aucprc_original :.2f}")
535 print(f"SMOTE: {lr_aucprc_smote:.2f}")
536 print(f"ADASYN: {lr_aucprc_adasyn:.2f}")
537
538 print("\nRandom Forest AUPRC:")
539 print(f"Original: {rf_aucprc_original :.2f}")
540 print(f"SMOTE: {rf_aucprc_smote:.2f}")
541 print(f"ADASYN: {rf_aucprc_adasyn:.2f}")
542
543 # 可视化结果
544 import matplotlib.pyplot as plt
545
546 methods = ['Original', 'SMOTE', 'ADASYN']
547 lr_aucprc_values = [lr_aucprc_original, lr_aucprc_smote, lr_aucprc_adasyn]
548 rf_aucprc_values = [rf_aucprc_original, rf_aucprc_smote, rf_aucprc_adasyn]
549
550 x = range(len(methods))
551
552 plt.figure(figsize=(10, 6))
553 plt.plot(x, lr_aucprc_values, marker='o', linestyle='-', color='b', label='Logistic
    Regression')
554 plt.plot(x, rf_aucprc_values, marker='o', linestyle='-', color='g', label='Random

```

```

Forest')
555
556 plt.xticks(x, methods)
557 plt.xlabel('Sampling Method')
558 plt.ylabel('AUPRC')
559 plt.title('AUPRC by Sampling Method and Model')
560 plt.legend()
561 plt.grid(True)
562
563 # 在点上显示具体数值
564 for i, txt in enumerate(lr_auprc_values):
565     plt.annotate(f'{txt :.2f}', (x[i], lr_auprc_values[i]), textcoords="offset points",
566                  xytext=(0, 5), ha='center')
567 for i, txt in enumerate(rf_auprc_values):
568     plt.annotate(f'{txt :.2f}', (x[i], rf_auprc_values[i]), textcoords="offset points",
569                  xytext=(0, 5), ha='center')
570
571 plt.show()
572
573 # %%
574 import pandas as pd
575 from sklearn.ensemble import IsolationForest
576 from sklearn.neighbors import LocalOutlierFactor
577 from sklearn.model_selection import train_test_split
578 from sklearn.preprocessing import StandardScaler
579 from sklearn.linear_model import LogisticRegression
580 from sklearn.ensemble import RandomForestClassifier
581 from sklearn.metrics import precision_recall_curve, auc
582
583 # 读取数据集
584
585 # 提取特征和标签
586 features = data.drop(columns=['Class'])
587 labels = data['Class']
588
589 # 分割数据集为训练集和测试集
590 X_train, X_test, y_train, y_test = train_test_split(features, labels, test_size=0.3,
591                                                     random_state=42, stratify=labels)

```

```
590
591 # 数据标准化
592 scaler = StandardScaler ()
593 X_train = scaler . fit_transform (X_train)
594 X_test = scaler . transform(X_test)
595
596 # 初始化孤立森林和局部异常因子
597 iso_forest = IsolationForest (contamination=0.01, random_state=42)
598 lof = LocalOutlierFactor (n_neighbors=20, contamination=0.01)
599
600 # 进行异常检测并剔除异常值
601 iso_forest . fit (X_train)
602 iso_outliers = iso_forest . predict (X_train) == 1
603
604 lof_outliers = lof . fit_predict (X_train) == 1
605
606 X_train_iso = X_train[ iso_outliers ]
607 y_train_iso = y_train[ iso_outliers ]
608
609 X_train_lof = X_train[ lof_outliers ]
610 y_train_lof = y_train[ lof_outliers ]
611
612 # 定义模型
613 logistic_regression = LogisticRegression (random_state=42, max_iter=100)
614 random_forest = RandomForestClassifier(n_estimators=50, max_depth=10, random_state
    =42)
615
616 # 训练和评估模型
617 def evaluate_model(model, X_train, y_train, X_test, y_test):
618     model. fit (X_train, y_train )
619     y_scores = model.predict_proba (X_test)[:, 1]
620     precision, recall, _ = precision_recall_curve (y_test, y_scores)
621     return auc( recall, precision )
622
623 # 原始数据上的AUPRC
624 lr_auprc_original = evaluate_model( logistic_regression, X_train, y_train, X_test,
    y_test )
625 rf_auprc_original = evaluate_model(random_forest, X_train, y_train, X_test, y_test)
626
```

```
627 # 孤立森林异常检测后的AUPRC
628 lr_auprc_iso = evaluate_model( logistic_regression , X_train_iso , y_train_iso , X_test,
        y_test )
629 rf_auprc_iso = evaluate_model(random_forest, X_train_iso , y_train_iso , X_test,
        y_test )
630
631 # 局部异常因子异常检测后的AUPRC
632 lr_auprc_lof = evaluate_model( logistic_regression , X_train_lof, y_train_lof , X_test,
        y_test )
633 rf_auprc_lof = evaluate_model(random_forest, X_train_lof, y_train_lof , X_test,
        y_test )
634
635 # 打印结果
636 print ("Logistic Regression AUPRC:")
637 print (f"Original : { lr_auprc_original :.2 f}")
638 print (f"Isolation Forest : { lr_auprc_iso :.2 f}")
639 print (f"Local Outlier Factor : { lr_auprc_lof :.2 f}")
640
641 print ("\nRandom Forest AUPRC:")
642 print (f"Original : { rf_auprc_original :.2 f}")
643 print (f"Isolation Forest : { rf_auprc_iso :.2 f}")
644 print (f"Local Outlier Factor : { rf_auprc_lof :.2 f}")
645
646 # 可视化结果
647 import matplotlib.pyplot as plt
648
649 methods = [ 'Original' , ' Isolation Forest' , 'Local Outlier Factor' ]
650 lr_auprc_values = [ lr_auprc_original , lr_auprc_iso , lr_auprc_lof ]
651 rf_auprc_values = [ rf_auprc_original , rf_auprc_iso , rf_auprc_lof ]
652
653 x = range(len(methods))
654
655 plt . figure ( figsize =(10, 6))
656 plt . plot (x, lr_auprc_values , marker='o' , linestyle ='-' , color='b' , label=' Logistic
        Regression' )
657 plt . plot (x, rf_auprc_values , marker='o' , linestyle ='-' , color='g' , label='Random
        Forest' )
658
659 plt . xticks (x, methods)
```



```
660 plt.xlabel('Anomaly Detection Method')
661 plt.ylabel('AUPRC')
662 plt.title('AUPRC by Anomaly Detection Method and Model')
663 plt.legend()
664 plt.grid(True)
665
666 # 在点上显示具体数值
667 for i, txt in enumerate(lr_auprc_values):
668     plt.annotate(f'{txt :.2 f}', (x[i], lr_auprc_values[i]), textcoords="offset points",
669                 xytext=(0, 5), ha='center')
670 for i, txt in enumerate(rf_auprc_values):
671     plt.annotate(f'{txt :.2 f}', (x[i], rf_auprc_values[i]), textcoords="offset points",
672                 xytext=(0, 5), ha='center')
673
674 plt.show()
675
676
677
678 # %% [markdown]
679 # # 1 划分数据集
680
681 # %%
682 data = pd.read_csv('data/creditcard.csv')
683
684 # %%
685 # 划分数据集
686 X = data.drop(['Class'], axis = 1).values
687 Y = data['Class'].values
688 del data
689
690 # %% [markdown]
691 # # 2 训练模型
692
693 # %%
694 from sklearn.model_selection import train_test_split, learning_curve
695 from sklearn import metrics
696
```

```

697 # %%
698 # 划分训练集与测试集
699 randomState = 10
700 np.random.seed(randomState)
701 X_dtrain, X_deval, y_dtrain, y_deval = train_test_split (X,
702                                                         Y,
703                                                         test_size = 0.3,
704                                                         random_state = randomState)
705
706 # %% [markdown]
707 # ## 2.1 Logistic 算法训练模型
708
709 # %%
710 # Logistic 分类
711 from sklearn.linear_model import LogisticRegression
712 # 网格搜索
713 from sklearn.grid_search import GridSearchCV
714
715 # %%
716 # 需搜索的参数
717 log_param_grid = {'C': list (np.linspace (0.05, 1, 20))} # 正则项系数的倒数，它必
# 须是一个正的浮点数
# 像支持向量机一样，值越
# 小则正则化更强
718
719 }
720 gsearch = GridSearchCV(
721     estimator = LogisticRegression (tol = 1e-8,
722                                     class_weight = 'balanced',
723                                     solver = 'sag',
724                                     max_iter = 500,
725                                     multi_class = 'ovr',
726                                     n_jobs = -1
727     ),
728     param_grid = log_param_grid,
729     cv = 5,
730     n_jobs = -1)
731
732 gsearch.fit (X_dtrain, y_dtrain.ravel ())
733

```

```

734 # 打印每个搜索的分数
735 print('grid scores :\n')
736 for i in gsearch.grid_scores_:
737     print(i)
738 # 打印最佳参数
739 print('\nbest params : ', gsearch.best_params_)
740 # 打印最高分数
741 print('\nbest score : ', gsearch.best_score_)
742
743 # %%
744 clf_log = LogisticRegression(tol = 1e-8,
745                               C = 0.35,
746                               class_weight = 'balanced',
747                               solver = 'sag',
748                               max_iter = 500,
749                               multi_class = 'ovr',
750                               n_jobs = -1)
751 clf_log.fit(X_dtrain, y_dtrain)
752 y_pred = clf_log.predict(X_deval)
753 print('Logistic Accuracy : ', metrics.accuracy_score(y_deval, y_pred))
754 print('Logistic Recall : ', metrics.recall_score(y_deval, y_pred))
755 print('Logistic Precision : ', metrics.precision_score(y_deval, y_pred))
756 print('Logistic F1 score : ', metrics.f1_score(y_deval, y_pred))
757
758 # %% [markdown]
759 # ## 2.2 SVM算法训练模型
760
761 # %%
762 # 支持向量机分类
763 from sklearn.svm import SVC
764
765 # %%
766 # 网格搜索
767 from sklearn.grid_search import GridSearchCV
768 # 测试C参数
769 SVM_param_grid = {'C': list(range(1, 10))}
770 gsearch = GridSearchCV(
771     estimator = SVC(tol = 1e-8,
772                     class_weight = 'balanced',

```

```
773         max_iter = 500),
774     param_grid = SVM_param_grid,
775     cv = 5,
776     n_jobs = -1)
777 gsearch.fit(X_dtrain, y_dtrain.ravel())
778
779 # 打印每个搜索的分数
780 print('grid scores :')
781 for i in gsearch.grid_scores_:
782     print(i)
783 # 打印最佳参数
784 print('\nbest params : ', gsearch.best_params_)
785 # 打印最高分数
786 print('\nbest score : ', gsearch.best_score_)
787
788 # %%
789 # 测试kernel参数
790 SVM_param_grid = {'kernel': ['rbf', 'linear', 'poly', 'sigmoid']}
791 gsearch = GridSearchCV(
792     estimator = SVC(C = 1.0,
793                     tol = 1e-8,
794                     class_weight = 'balanced',
795                     max_iter = 500),
796     param_grid = SVM_param_grid,
797     cv = 5,
798     n_jobs = -1)
799 gsearch.fit(X_dtrain, y_dtrain.ravel())
800
801 # 打印每个搜索的分数
802 print('grid scores :')
803 for i in gsearch.grid_scores_:
804     print(i)
805 # 打印最佳参数
806 print('\nbest params : ', gsearch.best_params_)
807 # 打印最高分数
808 print('\nbest score : ', gsearch.best_score_)
809
810 # %%
811 # 测试gamma参数
```

```

812 SVM_param_grid = {'gamma': list(np.linspace ((1/30) , 1, 30))}
813 gsearch = GridSearchCV(
814     estimator = SVC(C = 1.0,
815                     kernel = 'poly',
816                     tol = 1e-8,
817                     class_weight = 'balanced',
818                     max_iter = 500),
819     param_grid = SVM_param_grid,
820     cv = 5,
821     n_jobs = -1)
822 gsearch.fit(X_dtrain, y_dtrain.ravel())
823
824 # 打印每个搜索的分数
825 print('grid scores :')
826 for i in gsearch.grid_scores_:
827     print(i)
828 # 打印最佳参数
829 print('\nbest params : ', gsearch.best_params_)
830 # 打印最高分数
831 print('\nbest score : ', gsearch.best_score_)
832
833 # %%
834 clf_SVM = SVC(C = 2.0,
835               kernel = 'poly',
836               gamma = 1/6,
837               tol = 1e-8,
838               class_weight = 'balanced',
839               max_iter = 500)
840 clf_SVM.fit(X_dtrain, y_dtrain)
841 y_pred = clf_SVM.predict(X_deval)
842 print('SVM Accuracy : ', metrics.accuracy_score(y_deval, y_pred))
843 print('SVM Recall : ', metrics.recall_score(y_deval, y_pred))
844 print('SVM Precision : ', metrics.precision_score(y_deval, y_pred))
845 print('SVM F1 score : ', metrics.f1_score(y_deval, y_pred))
846
847 # %% [markdown]
848 # ## 2.3 RandomForest算法训练模型
849
850 # %%

```

```
851 # 随机森林分类器
852 from sklearn.ensemble import RandomForestClassifier
853
854 # %%
855 # 网格搜索
856 from sklearn.grid_search import GridSearchCV
857 # 测试n_estimators参数
858 tree_param_grid = {'n_estimators': list(range(100, 1100, 100))} # 树变量
859 gsearch = GridSearchCV(
860     estimator = RandomForestClassifier(oob_score = True,
861                                     n_jobs = -1,
862                                     random_state = 10),
863     param_grid = tree_param_grid,
864     cv = 5,
865     n_jobs = -1)
866 gsearch.fit(X_dtrain, y_dtrain.ravel())
867
868 # 打印每个搜索的分数
869 print('grid scores :')
870 for i in gsearch.grid_scores_:
871     print(i)
872 # 打印最佳参数
873 print('\nbest params : ', gsearch.best_params_)
874 # 打印最高分数
875 print('\nbest score : ', gsearch.best_score_)
876
877 # %%
878 # 测试max_depth参数
879 tree_param_grid = {'max_depth': list(range(3, 10))} # 树变量
880 gsearch = GridSearchCV(
881     estimator = RandomForestClassifier(n_estimators = 300,
882                                     oob_score = True,
883                                     n_jobs = -1,
884                                     random_state = 5),
885     param_grid = tree_param_grid,
886     cv = 5,
887     n_jobs = -1)
888 gsearch.fit(X_dtrain, y_dtrain.ravel())
889
```

```
890 # 打印每个搜索的分数
891 print('grid scores :')
892 for i in gsearch.grid_scores_ :
893     print(i)
894 # 打印最佳参数
895 print('\nbest params : ', gsearch.best_params_)
896 # 打印最高分数
897 print('\nbest score : ', gsearch.best_score_)
898
899 # %%
900
901
902 # %%
903 # 测试max_features参数
904 tree_param_grid = {'max_features': list(range(3, 30))} # 树变量
905 gsearch = GridSearchCV(
906     estimator = RandomForestClassifier(n_estimators = 300,
907                                     max_depth = 5,
908                                     oob_score = True,
909                                     n_jobs = -1,
910                                     random_state = 5),
911     param_grid = tree_param_grid,
912     cv = 5,
913     n_jobs = -1)
914 gsearch.fit(X_dtrain, y_dtrain.ravel())
915
916 # 打印每个搜索的分数
917 print('grid scores :')
918 for i in gsearch.grid_scores_ :
919     print(i)
920 # 打印最佳参数
921 print('\nbest params : ', gsearch.best_params_)
922 # 打印最高分数
923 print('\nbest score : ', gsearch.best_score_)
924
925 # %%
926 clf_rf = RandomForestClassifier(n_estimators = 300,
927                                max_depth = 5,
928                                oob_score = True,
```

```

929         n_jobs = -1,
930         random_state = 5)
931 clf_rf . fit (X_dtrain, y_dtrain)
932 y_pred = clf_rf . predict (X_deval)
933 print ( 'RandomForest Accuracy : ', metrics . accuracy_score(y_deval, y_pred))
934 print ( 'RandomForest Recall    : ', metrics . recall_score (y_deval, y_pred))
935 print ( 'RandomForest Precision : ', metrics . precision_score (y_deval, y_pred))
936 print ( 'RandomForest F1 score  : ', metrics . f1_score(y_deval, y_pred))
937
938 # %% [markdown]
939 # ## 2.4 xgboost算法训练模型
940
941 # %%
942 import operator
943 import xgboost as xgb
944
945 # %%
946 # 平衡数据处理
947 weights = ((Y == 0).sum() / (1.0 * (Y == 1).sum()))
948 print ( '平衡数据调整权重: ', weights)
949
950 # %%
951 # 网格搜索
952 from sklearn . grid_search import GridSearchCV
953
954 # 需搜索的参数
955 tree_param_grid = { 'n_estimators': list (range(100, 1100, 100)) # 树的个数
956                    }
957 gsearch = GridSearchCV(
958     estimator = xgb.XGBClassifier( silent = 1,
959                                     scale_pos_weight = weights,
960                                     n_jobs = -1,
961                                     eval_metric = 'auc'
962                                ),
963     param_grid = tree_param_grid,
964     cv = 5,
965     n_jobs = -1)
966
967 gsearch . fit (X_dtrain, y_dtrain . ravel ())

```



```
968
969 # 打印每个搜索的分数
970 print('grid scores :\n')
971 for i in gsearch.grid_scores_:
972     print(i)
973 # 打印最佳参数
974 print('\nbest params : ', gsearch.best_params_)
975 # 打印最高分数
976 print('\nbest score : ', gsearch.best_score_)
977
978 # %%
979 # 需搜索的参数
980 tree_param_grid = {'max_depth': list(range(3, 10))    # 树的深度, 越大越容易过拟
981                    }
982 gsearch = GridSearchCV(
983     estimator = xgb.XGBClassifier(silent = 1,
984                                   scale_pos_weight = weights,
985                                   n_jobs = -1,
986                                   eval_metric = 'auc',
987                                   n_estimators = 400
988                                   ),
989     param_grid = tree_param_grid,
990     cv = 5,
991     n_jobs = -1)
992
993 gsearch.fit(X_dtrain, y_dtrain.ravel())
994
995 # 打印每个搜索的分数
996 print('grid scores :\n')
997 for i in gsearch.grid_scores_:
998     print(i)
999 # 打印最佳参数
1000 print('\nbest params : ', gsearch.best_params_)
1001 # 打印最高分数
1002 print('\nbest score : ', gsearch.best_score_)
1003
1004 # %%
1005 tree_param_grid = {'min_child_weight': list(range(1, 10))    # min_child_weight = 1
```

```

1006                                     # 即叶子节点中最少需
1007                                     要包含 100 个样本
1008                                     # 参数值越小，越容易
1009                                     overfitting
1010
1011     }
1012 gsearch = GridSearchCV(
1013     estimator = xgb.XGBClassifier( silent = 1,
1014                                     scale_pos_weight = weights,
1015                                     n_jobs = -1,
1016                                     eval_metric = 'auc',
1017                                     n_estimators = 400,
1018                                     max_depth = 6
1019                                     ),
1020     param_grid = tree_param_grid,
1021     cv = 5,
1022     n_jobs = -1)
1023
1024 gsearch.fit(X_dtrain, y_dtrain.ravel())
1025
1026 # 打印每个搜索的分数
1027 print('grid scores :\n')
1028 for i in gsearch.grid_scores_:
1029     print(i)
1030 # 打印最佳参数
1031 print('\nbest params : ', gsearch.best_params_)
1032 # 打印最高分数
1033 print('\nbest score : ', gsearch.best_score_)
1034
1035 # %%
1036 min_child_weight = list(gsearch.best_params_.values())[0]
1037 tree_param_grid = {'subsample': list(np.linspace(0.5, 1, 6)) # 列采样占比，通常0
1038                    .5~1
1039                    }
1040 gsearch = GridSearchCV(
1041     estimator = xgb.XGBClassifier( silent = 1,
1042                                     scale_pos_weight = weights,
1043                                     n_jobs = -1,
1044                                     eval_metric = 'auc',
1045                                     n_estimators = 400,

```

```

1042         max_depth = 6,
1043         min_child_weight = min_child_weight
1044     ),
1045     param_grid = tree_param_grid,
1046     cv = 5,
1047     n_jobs = -1)
1048
1049 gsearch.fit(X_dtrain, y_dtrain.ravel())
1050
1051 # 打印每个搜索的分数
1052 print('grid scores :\n')
1053 for i in gsearch.grid_scores_:
1054     print(i)
1055 # 打印最佳参数
1056 print('\nbest params : ', gsearch.best_params_)
1057 # 打印最高分数
1058 print('\nbest score : ', gsearch.best_score_)
1059
1060 # %%
1061 subsample = list(gsearch.best_params_.values())[0]
1062 tree_param_grid = {'colsample_bytree': list(np.linspace(0.5, 1, 6)) # 生成树时进
1063                    # 行的特征采样占比，通常0.5~1
1064                    }
1065 gsearch = GridSearchCV(
1066     estimator = xgb.XGBClassifier(silent = 1,
1067     scale_pos_weight = weights,
1068     n_jobs = -1,
1069     eval_metric = 'auc',
1070     n_estimators = 400,
1071     max_depth = 6,
1072     min_child_weight = min_child_weight,
1073     subsample = subsample
1074 ),
1075     param_grid = tree_param_grid,
1076     cv = 5,
1077     n_jobs = -1)
1078 gsearch.fit(X_dtrain, y_dtrain.ravel())
1079

```

```
1080 # 打印每个搜索的分数
1081 print('grid scores :\n')
1082 for i in gsearch.grid_scores_ :
1083     print(i)
1084 # 打印最佳参数
1085 print('\nbest params : ', gsearch.best_params_)
1086 # 打印最高分数
1087 print('\nbest score : ', gsearch.best_score_)
1088
1089 # %%
1090 colsample_bytree = list(gsearch.best_params_.values())[0]
1091 tree_param_grid = {'reg_alpha': list(range(1, 10)) # 权重值的L1正则化项参数
1092                    }
1093 gsearch = GridSearchCV(
1094     estimator = xgb.XGBClassifier(silent = 1,
1095                                   scale_pos_weight = weights,
1096                                   n_jobs = -1,
1097                                   eval_metric = 'auc',
1098                                   n_estimators = 400,
1099                                   max_depth = 6,
1100                                   min_child_weight = min_child_weight,
1101                                   subsample = subsample,
1102                                   colsample_bytree = colsample_bytree
1103                                   ),
1104     param_grid = tree_param_grid,
1105     cv = 5,
1106     n_jobs = -1)
1107
1108 gsearch.fit(X_dtrain, y_dtrain.ravel())
1109
1110 # 打印每个搜索的分数
1111 print('grid scores :\n')
1112 for i in gsearch.grid_scores_ :
1113     print(i)
1114 # 打印最佳参数
1115 print('\nbest params : ', gsearch.best_params_)
1116 # 打印最高分数
1117 print('\nbest score : ', gsearch.best_score_)
1118
```

```
1119 # %%
1120 reg_alpha = list(gsearch.best_params_.values())[0]
1121 tree_param_grid = {'reg_lambda': list(range(1, 10)) # 权重值的L2正则化项参数
1122                  }
1123 gsearch = GridSearchCV(
1124     estimator = xgb.XGBClassifier(silent = 1,
1125                                   scale_pos_weight = weights,
1126                                   n_jobs = -1,
1127                                   eval_metric = 'auc',
1128                                   n_estimators = 400,
1129                                   max_depth = 6,
1130                                   min_child_weight = min_child_weight,
1131                                   subsample = subsample,
1132                                   colsample_bytree = colsample_bytree,
1133                                   reg_alpha = reg_alpha
1134                               ),
1135     param_grid = tree_param_grid,
1136     cv = 5,
1137     n_jobs = -1)
1138
1139 gsearch.fit(X_dtrain, y_dtrain.ravel())
1140
1141 # 打印每个搜索的分数
1142 print('grid scores :\n')
1143 for i in gsearch.grid_scores_:
1144     print(i)
1145 # 打印最佳参数
1146 print('\nbest params : ', gsearch.best_params_)
1147 # 打印最高分数
1148 print('\nbest score : ', gsearch.best_score_)
1149
1150 # %%
1151 reg_lambda = list(gsearch.best_params_.values())[0]
1152 tree_param_grid = {'seed': list(range(0, 1001, 100)) # 随机种子
1153                  }
1154 gsearch = GridSearchCV(
1155     estimator = xgb.XGBClassifier(silent = 1,
1156                                   scale_pos_weight = weights,
1157                                   n_jobs = -1,
```

```
1158         eval_metric = 'auc',
1159         n_estimators = 400,
1160         max_depth = 6,
1161         min_child_weight = min_child_weight,
1162         subsample = subsample,
1163         colsample_bytree = colsample_bytree,
1164         reg_alpha = reg_alpha,
1165         reg_lambda = reg_lambda
1166     ),
1167     param_grid = tree_param_grid,
1168     cv = 5,
1169     n_jobs = -1)
1170
1171 gsearch.fit(X_dtrain, y_dtrain.ravel())
1172
1173 # 打印每个搜索的分数
1174 print('grid scores :\n')
1175 for i in gsearch.grid_scores_:
1176     print(i)
1177 # 打印最佳参数
1178 print('\nbest params : ', gsearch.best_params_)
1179 # 打印最高分数
1180 print('\nbest score : ', gsearch.best_score_)
1181
1182 # %%
1183 seed = list(gsearch.best_params_.values())[0]
1184 # 训练XGBoost模型
1185 model = xgb.XGBClassifier(silent = 1,
1186                           scale_pos_weight = weights,
1187                           n_jobs = -1,
1188                           eval_metric = 'auc',
1189                           n_estimators = 400,
1190                           max_depth = 6,
1191                           min_child_weight = min_child_weight,
1192                           subsample = subsample,
1193                           colsample_bytree = colsample_bytree,
1194                           reg_alpha = reg_alpha,
1195                           reg_lambda = reg_lambda,
1196                           seed = seed
```

```

1197         )
1198 y_pred = model.fit(X_dtrain, y_dtrain).predict(X_deval)
1199 print('XGBoost Accuracy : ', metrics.accuracy_score(y_deval, y_pred))
1200 print('XGBoost Recall : ', metrics.recall_score(y_deval, y_pred))
1201 print('XGBoost Precision : ', metrics.precision_score(y_deval, y_pred))
1202 print('XGBoost F1 score : ', metrics.f1_score(y_deval, y_pred))
1203
1204 # %%
1205 # 确定特征因子重要性
1206 plt.rcParams['font.sans-serif'] = ['SimHei']
1207 plt.rcParams['axes.unicode_minus'] = False
1208
1209 # 绘制特征重要性评分
1210 colours = plt.cm.Set1(np.linspace(0, 1, 30))
1211 fig = plt.figure(figsize=(16, 10))
1212 ax = fig.add_subplot(111)
1213 ax = xgb.plot_importance(model,
1214                          height=0.8,
1215                          color=colours,
1216                          grid=False,
1217                          show_values=True,
1218                          importance_type='cover',
1219                          ax=ax)
1220 for axis in ['top', 'bottom', 'left', 'right']:
1221     ax.spines[axis].set_linewidth(2)
1222
1223 ax.set_xlabel('重要性评分', size=16)
1224 ax.set_ylabel('特征', size=16)
1225 ax.set_yticklabels(ax.get_yticklabels(), size=12)
1226 ax.set_title('模型学习后特征重要性排序', size=20)
1227
1228 # %%
1229 # 模型重要性的决策树可视化
1230 plt.figure(figsize=(12, 8))
1231 os.environ['PATH'] += os.pathsep + 'D:/Graphviz2.38/bin'
1232 xgb.to_graphviz(model)
1233
1234 # %%
1235 # 学习曲线显示精度

```

```

1236 trainSizes , trainScores , crossValScores = learning_curve (
1237     xgb.XGBClassifier( silent = 1,
1238         scale_pos_weight = weights,
1239         n_jobs = -1,
1240         eval_metric = 'auc',
1241         n_estimators = 400,
1242         max_depth = 6,
1243         min_child_weight = min_child_weight,
1244         subsample = subsample,
1245         colsample_bytree = colsample_bytree,
1246         reg_alpha = reg_alpha,
1247         reg_lambda = reg_lambda,
1248         seed = seed
1249     ),
1250     X_dtrain,
1251     y_dtrain ,
1252     cv = 5,
1253     scoring = 'accuracy'
1254 )
1255
1256 # %%
1257 trainScoresMean = np.mean(trainScores , axis = 1)
1258 trainScoresStd = np.std( trainScores , axis = 1)
1259 crossValScoresMean = np.mean(crossValScores, axis = 1)
1260 crossValScoresStd = np.std( crossValScores , axis = 1)
1261
1262 fig = plt . figure ( figsize = (12, 8))
1263 plt . fill_between ( trainSizes ,
1264     trainScoresMean - trainScoresStd ,
1265     trainScoresMean + trainScoresStd ,
1266     alpha = 0.1,
1267     color = 'b')
1268 plt . fill_between ( trainSizes ,
1269     crossValScoresMean - crossValScoresStd ,
1270     crossValScoresMean + crossValScoresStd ,
1271     alpha = 0.1,
1272     color = 'r')
1273 plt . plot ( trainSizes ,
1274     trainScores .mean(axis = 1),

```



```

1275         'o-',
1276         label = 'TRAIN',
1277         color = 'b')
1278 plt.plot( trainSizes ,
1279           crossValScores.mean(axis = 1),
1280           'o-',
1281           label = 'CROSS-VAL',
1282           color = 'r')
1283
1284 ax = plt.gca()
1285 for axis in [ 'top', 'bottom', 'left', 'right']:
1286     ax.spines[ axis ]. set_linewidth (2)
1287
1288 handles, labels = ax.get_legend_handles_labels ()
1289 plt.legend(handles,
1290            [ 'TRAIN', 'CROSS-VAL'],
1291            bbox_to_anchor = (0.8, 0.15),
1292            loc = 2,
1293            borderaxespad = 0,
1294            fontsize = 16)
1295 plt.xlabel( 'TRAIN', size = 16)
1296 plt.ylabel( 'AUC', size = 16)
1297 plt.title( 'Learning Curves (70% samples)', size = 20)
1298
1299 # %%
1300 import matplotlib.pyplot as plt
1301
1302 # XGBoost模型超参数调优结果
1303 xgb_estimators = [100, 200, 300, 400, 500, 600, 700, 800, 900, 1000]
1304 xgb_auprc = [0.9876, 0.9952, 0.9981, 0.9988, 0.9990, 0.9992, 0.9993, 0.9994, 0.9995,
1305              0.9995]
1306
1307 # Logistic 回归模型超参数调优结果
1308 logistic_C = [0.05, 0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4, 0.45, 0.5, 0.55, 0.6,
1309               0.65, 0.7, 0.75, 0.8, 0.85, 0.9, 0.95, 1.0]
1310 logistic_auprc = [0.0012, 0.3247, 0.3401, 0.6012, 0.4579, 0.5018, 0.3321, 0.4895,
1311                  0.3338, 0.4827, 0.1637, 0.7261, 0.3382, 0.7423, 0.5024, 0.3214, 0.5031, 0.5019,
1312                  0.1687, 0.7271]

```

```
1310 # 随机森林模型超参数调优结果
1311 rf_estimators = [100, 200, 300, 400, 500, 600, 700, 800, 900, 1000]
1312 rf_aucprc = [0.9823, 0.9910, 0.9935, 0.9948, 0.9952, 0.9958, 0.9961, 0.9963, 0.9964,
1313             0.9965]
1314
1315 # 绘制XGBoost模型超参数调优结果
1316 plt.figure(figsize=(12, 8))
1317
1318 plt.subplot(3, 1, 1)
1319 plt.plot(xgb_estimators, xgb_aucprc, marker='o', linestyle='--', color='blue')
1320 plt.title('XGBoost AUPRC vs. n_estimators')
1321 plt.xlabel('n_estimators')
1322 plt.ylabel('AUPRC')
1323 plt.grid(True)
1324
1325 # 绘制Logistic回归模型超参数调优结果
1326 plt.subplot(3, 1, 2)
1327 plt.plot(logistic_C, logistic_aucprc, marker='o', linestyle='--', color='green')
1328 plt.title('Logistic Regression AUPRC vs. C')
1329 plt.xlabel('C')
1330 plt.ylabel('AUPRC')
1331 plt.grid(True)
1332
1333 # 绘制随机森林模型超参数调优结果
1334 plt.subplot(3, 1, 3)
1335 plt.plot(rf_estimators, rf_aucprc, marker='o', linestyle='--', color='red')
1336 plt.title('Random Forest AUPRC vs. n_estimators')
1337 plt.xlabel('n_estimators')
1338 plt.ylabel('AUPRC')
1339 plt.grid(True)
1340
1341 plt.tight_layout()
1342 plt.show()
1343
1344 # %%
```